



**ENHANCING AHO-CORASICK ALGORITHM WITH CONTEXT-AWARE AND
PRIORITY-WEIGHTED APPROXIMATE MATCHING APPLIED IN
ROBUST EXTRACTION OF MEDICAL TERMS
IN PATIENT NOTES**

A Thesis Presented to the
Faculty of the Computer Science Department
College of Information Systems and Technology Management
Pamantasan ng Lungsod ng Maynila
(University of the City of Manila)
Intramuros, Manila

In Partial Fulfillment of the Requirements for the Degree
Bachelor of Science in Computer Science (BSCS)

By:

MATANGA, SOPHIA VIEN V.
YUPANO, DENESSE ADRIELLE P.

Vivien A. Agustin
Adviser

August 2026



APPROVAL SHEET

The thesis hereto titled

**ENHANCING AHO-CORASICK ALGORITHM WITH CONTEXT-AWARE AND
PRIORITY-WEIGHTED APPROXIMATE MATCHING APPLIED IN
ROBUST EXTRACTION OF MEDICAL TERMS
IN PATIENT NOTES**

prepared and submitted by **Sophia Vien V. Matanga** and **Denesse Adrielle P. Yupano** in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science has been examined and is recommended for acceptance and approval for ORAL EXAMINATION.

Prof. Raymund M. Dioses
Coordinator

Prof. Vivien A. Agustin
Adviser

PANEL OF EXAMINERS

Approved by the Committee on Oral Examination
with a grade of _____ on _____.

Leisyl M. Mahusay
Panel Member

Marilou B. Mangrobang
Panel Member

Accepted and approved in partial fulfillment of the requirements for the degree of
Bachelor of Science in Computer Science.

Raymund M. Dioses
CS Chairperson
College of Information Systems and
Technology Management

Dr. Khatalyn E. Mata
Dean
College of Information Systems and
Technology Management



TABLE OF CONTENTS

TITLE PAGE.....	i
APPROVAL SHEET.....	ii
TABLE OF CONTENTS.....	iii
LIST OF FIGURES.....	iv
LIST OF TABLES.....	v
Chapter ONE.....	1
1.1 Background of the Study.....	1
1.2 Statement of the Problem.....	4
1.3 Objective of the Study.....	9
1.4 Significance of the Study.....	10
1.5 Scope and Limitations.....	11
1.6 Definition of Terms.....	13
Chapter TWO.....	16
2.1 Related Literature.....	16
2.2 Related Studies.....	25
2.3 Research Synthesis.....	32
Chapter THREE.....	36
3.1 Research Design.....	36
3.2 Proposed Algorithm and System Architecture.....	37
3.2.1 Proposed Algorithm.....	37
3.2.2 System Architecture.....	42
3.3 System Requirements.....	43
3.3.1 Hardware Requirements.....	43
3.3.2 Software Requirements.....	44
3.4 Theories, Methods, and Tools.....	46
3.4.1 Theories.....	46
3.4.2 Methods and Tools.....	51
REFERENCES.....	59



LIST OF FIGURES

FIGURE No.	FIGURE TITLE	PAGE No.
Figure 1.1	Repeated Failure Link Chain Traversal During Search.....	5
Figure 1.2	Panel A - Node Memory Layout.....	7
Figure 1.3	Panel B - Memory Access Pattern During Search.....	8
Figure 1.4	Panel C - Cache Miss Rate vs. Pattern Count.....	9
Figure 1.5	Full Alphabet Transition Storage Per State Regardless of Actual Usage.....	10
Figure 3.1	System Process Flow.....	46
Figure 3.2	Aho-Corasick Automaton from patterns {ab, b, bab, bac, db, dd}.....	50
Figure 3.3	DFA Representation of State Transitions.....	51
Figure 3.4	Non-Contiguous addresses after insertion.....	52
Figure 3.5	Sparse and Dense Block Classification in Double-Array AC.....	53
Figure 3.6	Local Context Window Extraction Around a Matched Token.....	54



LIST OF TABLES

TABLE No.	TABLE TITLE	PAGE No.
Table 3.1	Hardware Specifications for System Implementation.....	44
Table 3.2	Software Specifications for System Implementation.....	45



Chapter ONE INTRODUCTION

1.1 Background of the Study

The digitization of healthcare has led to the adoption of Electronic Health Records (EHRs), creating vast repositories of unstructured clinical documents, including handwritten doctors' prescriptions that contain critical medical information such as drug names, dosages, and frequency instructions. Simultaneously, medical research is increasingly relying on analyzing this data to improve patient care and make new discoveries in order to enhance operational functionalities (Negro-Calduch et al., 2021). This trend creates an urgent demand for automated, high-throughput methods to transform these unstructured texts into structured, coded data. Techniques for Clinical Named Entity Recognition (CNER) are therefore the key technology for this transformation, serving as the central lead for tasks ranging from automatically finding and labeling medical terms like diseases and medications in text to population health analytics. The Aho-Corasick algorithm is a fundamental tool for this approach, as it can scan text and find all dictionary terms very quickly and accurately, as long as the spelling is perfect. This is particularly relevant in the context of handwritten doctors' prescriptions.

According to Kumar (2025), the standard Aho-Corasick algorithm fails when used on most real-world clinical texts. The core issue is its deterministic design for exact string matching, which is misaligned with the noisy, nuanced, and highly variable nature of clinical documentation. This causes two major problems: poor recall where it misses valid terms due to spelling errors or local abbreviations (for example, "nasea" for "nausea"), and poor precision where it incorrectly matches terms in the wrong context (for example identifying "cold" as an illness in "the patient had a cold compress"). Current fixes, like separate spell-checkers or context rules, are applied as disconnected



steps. This disconnected approach is inefficient and error-prone, especially when the text could match multiple medical terms at once.

This study aims to solve these problems by building a new, improved version of the Aho-Corasick algorithm. The goal is to create a unified system that can handle spelling variations and understand context during the search process, not after. To do this, the study will: modify the algorithm by constructing a precomputed skip table to enable fast and approximate transitions, enhance memory access efficiency by implementing a two-pass breadth-first search, and optimize memory consumption using a tiered hot/cold transition storage strategy.

The main contribution is a practical, more accurate tool for finding medical terms in patient records. By building context and spelling tolerance directly into a fast, dictionary-based search, this enhanced algorithm can improve the data quality used for critical tasks like medical research and patient safety monitoring.

Existing Algorithm

Input:

$P = \{p_1, p_2, \dots, p_n\}$

$T = t_1 \dots t_m$ // text

AC-Build(P):

create root q_0

for each pattern p in P :

insert p into trie via goto transitions $\delta(q, a)$

Problem # 2

mark terminal state with $O(q) \leftarrow O(q) \cup \{p\}$

queue $\leftarrow \emptyset$

for each $a \in \Sigma$ with $\delta(q_0, a)$ defined:

Problem # 3

$f(\delta(q_0, a)) \leftarrow q_0$

enqueue($\delta(q_0, a)$)

for each $a \in \Sigma$ with $\delta(q_0, a)$ undefined:

Problem # 3

$\delta(q_0, a) \leftarrow q_0$ // optional fallback shortcut

while queue not empty:



$v \leftarrow \text{dequeue}()$

for each $a \in \Sigma$ with $\delta(v, a)$ defined:

Problem # 3

$u \leftarrow \delta(v, a)$

$x \leftarrow f(v)$

while $\delta(x, a)$ undefined and $x \neq q_0$:

$x \leftarrow f(x)$

if $\delta(x, a)$ defined:

$f(u) \leftarrow \delta(x, a)$

else:

$f(u) \leftarrow q_0$

$O(u) \leftarrow O(u) \cup O(f(u))$ // inherit outputs

$\text{enqueue}(u)$

return automaton $(Q, \Sigma, \delta, q_0, F, f, O)$

AC-Search(T):

state $\leftarrow q_0$

for each symbol a in T:

while $\delta(\text{state}, a)$ undefined and $\text{state} \neq q_0$:

state $\leftarrow f(\text{state})$

Problem # 1

if $\delta(\text{state}, a)$ defined:

state $\leftarrow \delta(\text{state}, a)$

else:

state $\leftarrow q_0$

if $O(\text{state}) \neq \emptyset$:

report all patterns in $O(\text{state})$ ending at current index



1.2 Statement of the Problem

The Aho-Corasick algorithm faces significant performance lapses on modern hardware due to inefficiencies and excessive memory consumption. Primary issues include repeated failure link traversals that trigger redundant memory accesses and thread divergence in parallel environments, alongside scattered node allocation that causes frequent cache misses and pointer-chasing delays. Furthermore, the traditional automaton relies on full alphabet transition storage, which allocates memory for every possible symbol regardless of actual usage, leading to bloated data structures. These memory-intensive characteristics frequently exceed GPU constraints and undermine the theoretical efficiency of the algorithm during large-scale pattern matching.

1. Repeated Failure Link Chain Traversal During Search. Running Aho-Corasick in parallel on a GPU is limited due to its memory constraints. It can only handle a number of patterns that fit in the GPU memory and when it exceeds that limit it affects the performance of the search. Some parallel versions of Aho-Corasick remove the failure link to save memory and to allow more patterns to fit in the GPU. However, if more threads are added, it does not scale well and also struggle to maintain high performance (Abbas et al., 2024). Although Aho-Corasick guarantees theoretical performance, its speed still heavily relies on the internal data structure used to represent the automaton. In practice repeated traversals of failure links can significantly slow down the search, especially when the data structures are not well optimized (Kanda et al., 2024). Building on this line of GPU-based optimization, the parallel failureless Aho-Corasick (PFAC) algorithm was developed to extend the standard Aho-Corasick automaton for efficient parallelization on GPUs (Baloi et al., 2023). According to Gagniuc et al. (2025), PFAC eliminates the failure link mechanism present in the standard Aho-Corasick automaton and assigns each GPU thread to a byte of the input stream, transforming the algorithm's traversal from sequential to massively parallel and making it suitable for GPU-based execution.



Figure 1.1

Repeated Failure Link Chain Traversal During Search

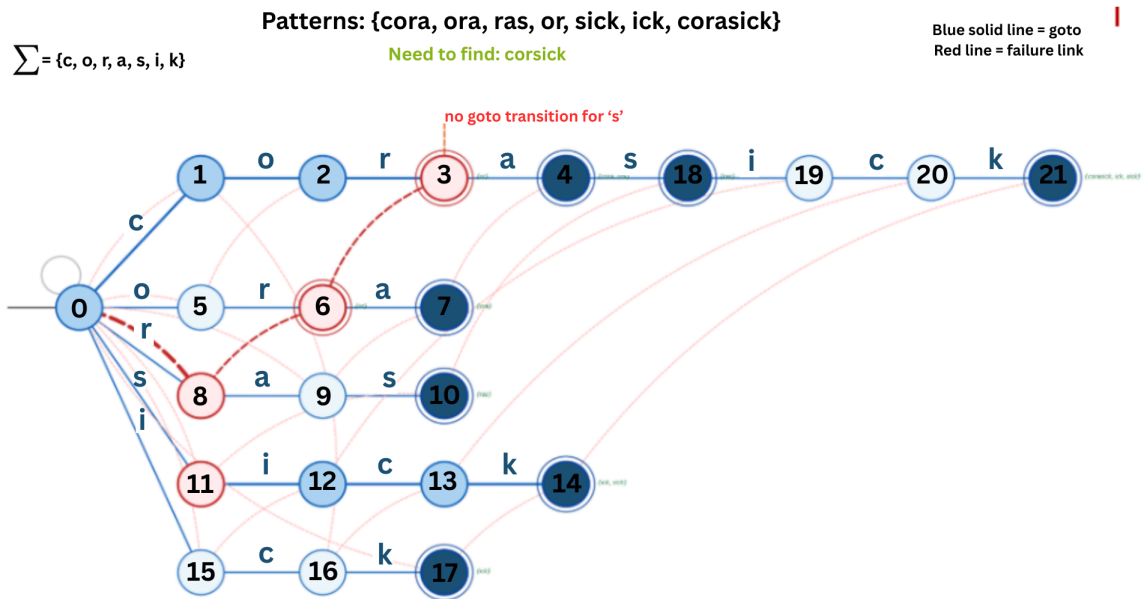


Figure 1.1 presents the Aho-Corasick automaton after building it from the patterns {cora, ora, ras, or, sick, ick, corasick}. The blue solid arrows are goto transitions and the red dashed arrows are failure links which is the fallback path taken when no valid transition exists for the current character. During search on the text "corsick", the algorithm successfully traverses $q_0 \rightarrow q_1(c) \rightarrow q_2(co) \rightarrow q_3(cor)$ using normal goto transitions. However, when the character 's' is encountered at q_3 , there is no valid goto transition for 's' at that state. Instead of restarting from the root, the algorithm must follow failure links one by one until it finds a state that has a valid transition for 's'. Each red dashed arrow in that chain is one sequential hop. Each step depends on the result of the previous one, so they cannot be executed at the same time. The algorithm must traverse failure links sequentially to determine the correct state. As the number of patterns increases, these chains become longer, which increases processing time particularly on GPUs that perform best with parallel operations.



2. Scattered Non-Contiguous Node Allocation Causing Cache Misses. During pattern detection, threads do not act depending on the access patterns, instead they access random locations in the AC transition tables based on the current input character that results in uncoalesced memory accesses (Çelebi & Yavanoğlu, 2023). Cuckoo trie explains that pointer-based trie is inefficient on modern hardware due to memory parallelism problems and cache miss. Since Aho-Corasick is a tri-based automaton it also faces a bottleneck in pointer-based trie and tree structure stems from dependent pointer-chasing DRAM accesses that cannot be effectively parallelized by modern processors (Zeitak & Morrison, 2022). Pointer-based structures like linked lists, skip lists, and trie-based automata such as Aho–Corasick suffer from poor spatial locality because their nodes are stored in scattered memory locations. This leads to frequent cache misses, which can significantly degrade performance even undermining theoretical efficiency advantages like logarithmic search time (Samyal et al., 2025).



Figure 1.2

Panel A - Node Memory Layout

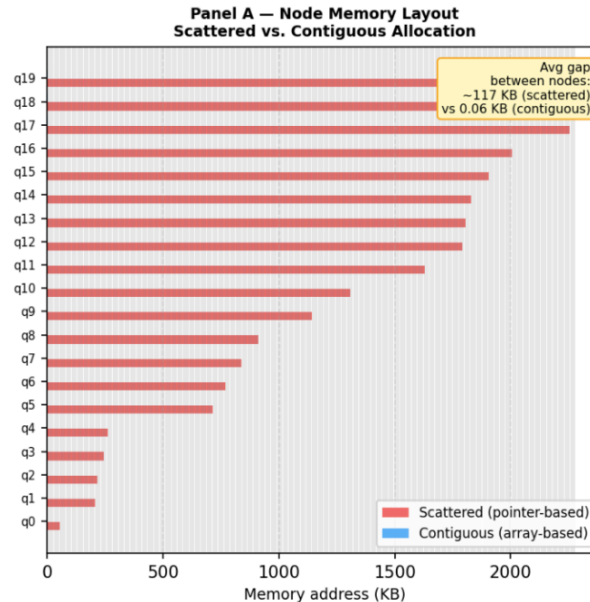


Figure 1.2 illustrates the cache miss problem from three angles. Panel A compares where trie nodes actually live in memory. The red bars (pointer-based, scattered) show nodes spread hundreds of kilobytes apart across the heap. The blue bars (contiguous) show nodes packed tightly together. This is the core structural problem, when the CPU needs to follow a pointer to the next trie node, it has to fetch from a completely different memory region, evicting whatever was in cache.



Figure 1.3

Panel B - Memory Access Pattern During Search

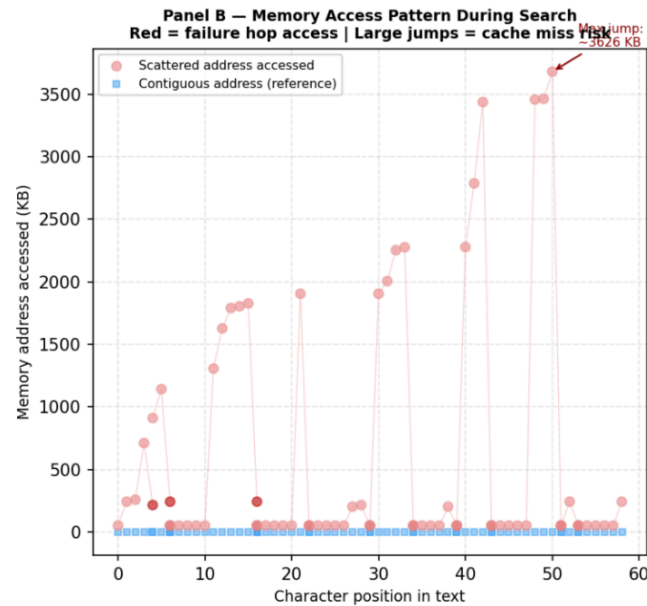


Figure 1.3 shows the memory access pattern during an actual search. The red line jumps wildly up and down, each spike is the CPU fetching a node from a distant heap location. The blue line stays flat near zero because contiguous nodes are always nearby in memory. The annotated spikes (miss) mark the worst cache miss events.



Figure 1.4

Panel C - Cache Miss Rate vs. Pattern Count

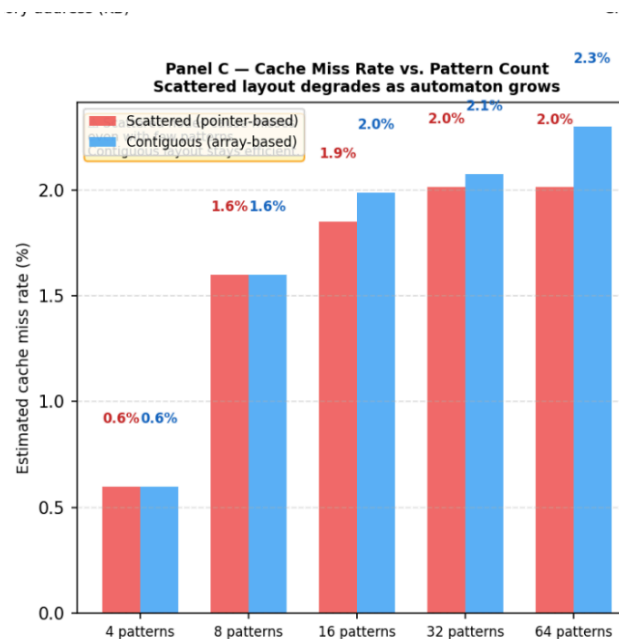


Figure 1.4 shows that this problem grows with pattern count. As more patterns are added, the automaton gets bigger and the scattered layout's miss rate climbs, while the contiguous layout stays consistently low.

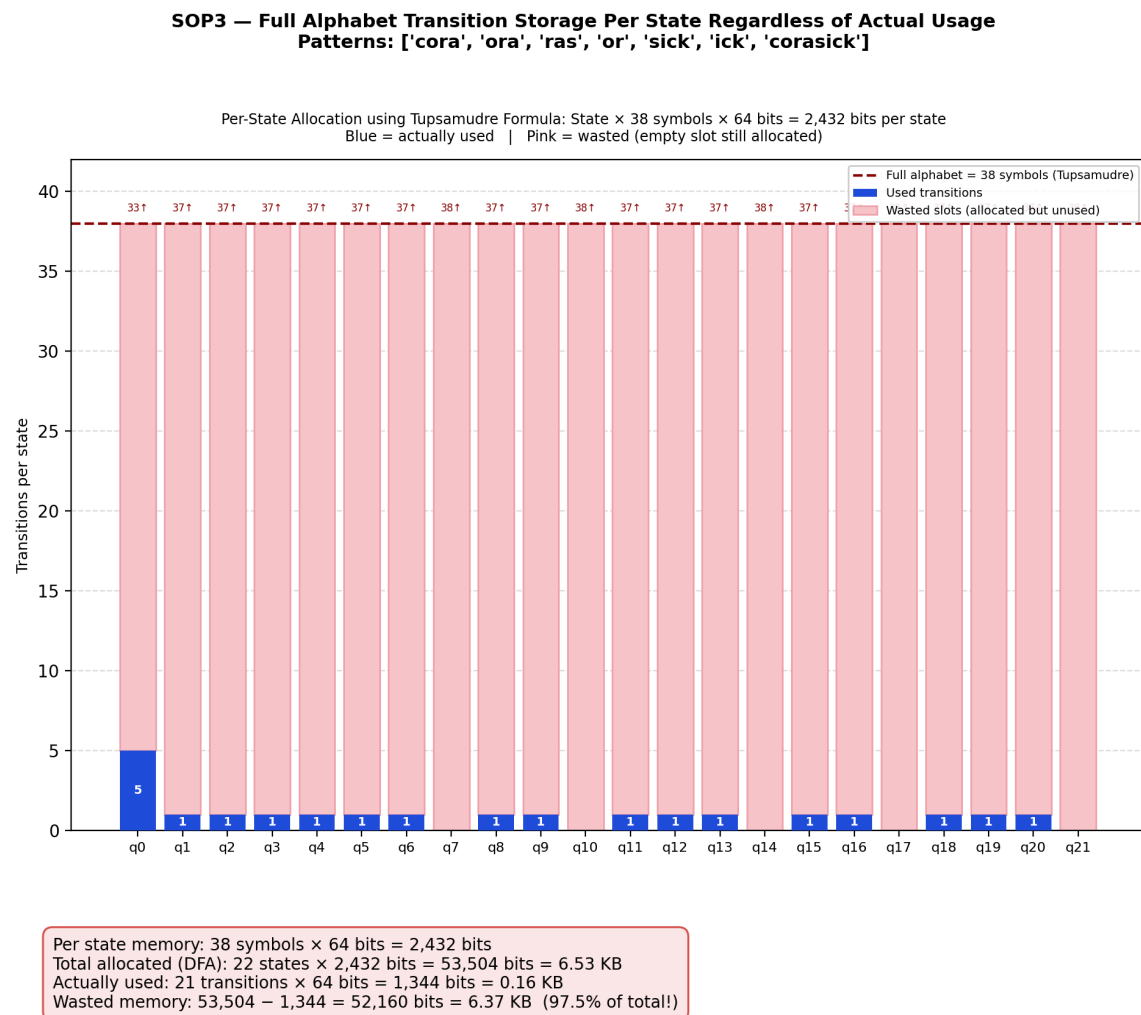
- 3. Full Alphabet Transition Storage Per State Regardless of Actual Usage.** In a traditional DFA like Aho-Corasick Algorithm, every possible transition from each state is stored separately in memory. As the number of pattern sets grows the number of state and transitions also increases leading to massive-size automata which causes cache space to be useless for large state transition tables and slows down the searching (Çelebi & Yavanoğlu, 2023). An integer occupies 64 bits and there are 38 numbers of symbols in the alphabet, the memory required to store the start state is $38 \times 64 \text{ bits} = 2,432 \text{ bits}$. In this unoptimized Aho-Corasick machine the memory is allocated for all possible symbol transitions from a given state, even if most of them are never used (Tupsamudre et al., 2021). The downside of Aho-Corasick algorithm is that it stores its automaton as a trie-based DFA and in



a classical DFA implementation, each state stores transitions for the full alphabet, even if it is an unused transition (BTSym 2021).

Figure 1.5

Full Alphabet Transition Storage Per State Regardless of Actual Usage



As shown in Figure 1.3, the memory allocation of the Aho–Corasick automaton is built from the patterns {cora, ora, ras, or, sick, ick, corasick}. Each bar represents one state of the automaton from q0 to q21. Because the alphabet size is 7, every state automatically reserves 7 transition slots for all possible characters. The blue sections represent the transitions that are actually used, while the pink sections represent the



transitions that were allocated but never used. Even if a state only needs one or two transitions, the system still allocates space for all 7 possible symbols, resulting in many unused slots.

As shown in the chart, most states use only one transition out of the 38 possible symbol slots allocated per state. Using the formula by Tupsamudre $38 \text{ symbols} \times 64 \text{ bits} = 2,432 \text{ bits per state}$, the automaton with 22 states allocates a total of 53,504 bits (6.53 KB) of memory. However, only 21 transition slots are actually used, which corresponds to just 1,344 bits (0.16 KB). This means 52,160 bits (6.37 KB) of allocated memory goes to waste resulting in approximately 97.5% of the total allocated memory being unused. This directly demonstrates how the classical DFA implementation of Aho-Corasick stores transitions for the full alphabet per state regardless of actual usage, causing significant memory overhead especially as the number of patterns and states grows larger.

1.3 Objective of the Study

This study aims to enhance the Aho-Corasick Algorithm applied in medical term extraction to effectively identify and match medical terminology across both correctly spelled and incorrectly spelled or locally abbreviated terms found in patient notes. This will be used to improve the accuracy of medical term recognition by eliminating repeated failure link traversal, optimizing memory layout for large medical dictionaries, and intelligently allocating transition storage, in order to support context-aware and priority-weighted approximate matching that distinguishes valid medical terms from ambiguous or misused terminology during clinical documentation processing activities.

1. To improve the pre-existing Aho-Corasick algorithm by constructing a precomputed Skip Table during the build phase that stores the direct destination state for every possible state and symbol combination, which will eliminate the repeated failure link chain traversal during the search phase, enforce a single constant-time lookup per character, and guarantee faster processing of medical terms in patient records regardless of spelling variations or text length.



2. To enhance memory access efficiency by reorganizing all trie nodes into contiguous memory locations using Two-Pass Breadth-First Search traversal order immediately after the trie is constructed, which will ensure that nodes sharing common medical prefixes are placed physically adjacent in memory, enable the CPU prefetcher to predict and preload the next memory address before it is needed, and reduce cache miss rates during the traversal of large medical dictionaries containing thousands of terms, abbreviations, and local variants.
3. To optimize memory consumption by implementing a Tiered Hot/Cold Transition Storage strategy that evaluates each state in the trie and classifies it as either a hot state with many transitions stored in a flat array for fast $O(1)$ access, or a cold state with few transitions stored in a hash map for memory-efficient storage, thereby allocating memory only where it is truly needed, reducing overall memory waste caused by full alphabet storage per state, and providing sufficient memory headroom to support the context-awareness layer that distinguishes ambiguous medical terms.

1.4 Significance of the Study

The result of this study will benefit the following:

For the General Public. Non-medical individuals involved in the care of family members or dependents stand to benefit from the outcomes of this study. Improved accuracy in medical term extraction directly enhances the quality of information provided by healthcare professionals. This study paves the way for tools that help non-medical individuals verify medication instructions by improving the extraction of drug names, dosages, and frequencies from handwritten prescriptions. This reduces the risk of dosage errors and improves medication adherence through clearer summaries.

For the Healthcare Industry and Clinical Information Practice. Healthcare institutions and developers of clinical decision systems stand to benefit from a more reliable tool for data dispensing. The proposed algorithm provides a layer of safety and



efficiency in the prescription-to-dispensing pipeline. By offering a matching mechanism that is robust against the “noise” of OCR-extracted text and the variations of medical abbreviations, the study supports pharmacists in cross-referencing handwritten notes with digital records more accurately, reducing manual transcription fatigue. By offering a method that is both more forgiving of documentation variations and more discerning of clinical context, this enhancement can improve the integrity of data used for critical tasks.

For the Field of Clinical Natural Language Processing (NLP) and Algorithm Design. This study provides a vital bridge between deterministic string matching and probabilistic OCR output. While the Aho-Corasick algorithm is efficient, it is notoriously brittle against the character-level errors common in OCR and “noise” from handwritten notes. This work contributes a blueprint for making dictionary-based matching fault-tolerant through context-aware and priority-weighted logic without losing the speed benefits of the Aho-Corasick structure.

For Future Researchers. This study establishes a foundational framework that opens a broader perspective for future work. Researchers can build upon this by expanding the dictionary to include symptoms or procedures, or by integrating the priority-weighted AC algorithm as a pre-processing filter. It serves as a benchmark for balancing computational efficiency with the messiness of real-world handwritten data.

The impact of this study goes beyond a single algorithm optimization. It provides a model for bridging the gap between legacy handwritten medical records and structured digital data. By introducing an intelligence layer to the AC algorithm, this study offers a solution that is both efficient for real-time use and resilient to the inherent noise of OCR-extracted text. Ultimately, this work supports the broader goals of patient safety by ensuring that the transition from a doctor’s pen to a digital system is accurate, prioritized, and contextually correct; laying the groundwork for a more reliable and data-driven clinical decision-making.



1.5 Scope and Limitations

This study covers the extraction of medical terms from handwritten doctors' prescriptions. Prior to the main algorithm Aho-Corasick, image processing is performed on the prescription image to extract raw text through Transformer-based Optical Character Recognition (TrOCR). The extracted text is then processed by Aho-Corasick algorithm, to handle approximate matching of medical terms such as drug names, dosages, and frequency instructions. The dataset of this study comes from self-collected handwritten prescriptions gathered from family members and friends, covering medical specialties such as general medicine, internal medicine, and pediatrics. The medical dictionary used in this study was developed based on The Philippine National Drug Formulary (PNDF) and RxNorm as authoritative sources for drug name standardization, while SNOMED CT is used for the representation of clinical treatments. Since there is no publicly available and standardized Philippine prescription dataset the research creates its own annotated corpus from locally collected prescriptions. These are then subjected to manual annotation to support the development and evaluation of a clinical named entity recognition (CNER) model to ensure consistency and accuracy. The technical focus of the research is limited to the algorithmic enhancements applied to the Aho-Corasick string matching core, specifically designed to improve robustness against lexical variations commonly found in prescription text, including spelling errors, abbreviations, and context-aware decoding. The evaluation focuses on the accuracy and efficiency of the enhanced matching mechanism in identifying medical terms. The study does not cover full end-to-end clinical information systems, electronic health record integration, or in-depth user interface design.

This study acknowledges several limitations regarding image processing and data coverage. The image processing component has limitations in handling poor quality prescription images. It may produce inaccurate results when handwriting is too distorted or illegible, and it cannot process prescriptions that contain only horizontal lines or dot-like marks, as these are filtered out during text detection and will return no extractable text. Additionally, the type of patient notes evaluated in this study is limited to handwritten doctors prescriptions only, clinical document types such as discharge



summaries, clinical notes, or laboratory results are out of the scope of the study. The medical dictionaries used in this study are taken only from publicly available medical sources on the internet rather than from actual clinical records from hospitals or other healthcare institutions. As a result, the data may not fully reflect the documentation practices, terminology variations, and formatting conventions present in real world clinical environments, which may affect the generalizability of the findings to real-world healthcare environments. The study also acknowledges limitations in the algorithmic and dictionary aspects of the system. The approach remains dictionary-dependent and cannot identify novel terms not present in the supplied resources. While context-aware, the model may not capture deeply complex clinical logic or long-range dependencies as effectively as a full-scale network model might, representing a trade-off for interpretability and runtime efficiency. The evaluation will be conducted on a limited set of specific medical concept types, namely drug names, dosages, and frequency instructions found in common outpatient prescriptions covering general medicine, internal medicine, and pediatrics, as these are the most frequently occurring terms in the collected prescriptions and are well-supported by the constructed medical dictionary. Finally, generalizability across other clinical concept types such as anatomical terms, procedures, symptoms, and laboratory findings, as well as broader clinical sub-languages and note types, cannot be guaranteed. The computational overhead of the added layers will also be a point of analysis compared to the baseline algorithm.

1.6 Definition of Terms

This section provides definitions for important terminology used in the study. The definitions are provided so that readers can comprehend the terms as they have been utilized in this study.

Aho-Corasick Algorithm / AC - The baseline string-searching automaton used in this study to locate medical patterns within patient notes; it is the specific algorithm being optimized through skip tables and tiered storage to eliminate failure link traversals and reduce memory usage.



Approximate Matching - The enhanced search capability introduced in this study that allows the algorithm to identify medical terms even when they contain character variations, specifically addressing the problem of misspelled clinical documentation.

Clinical Named Entity Recognition / CNER - The primary task of this research, referring to the automated process of identifying and categorizing specific medical entities; such as diseases, medications, and procedures.

Clinical Natural Language Processing / NLP - The specialized application of computational linguistics used in this study to parse unstructured physician notes and discharge summaries; it serves as the framework where the enhanced Aho-Corasick algorithm acts as the core engine for transforming raw medical text into structured, coded data for patient safety and research analytics.

Context-Aware Validation - The filtering layer integrated into the enhanced algorithm that analyzes surrounding text to distinguish between identical strings with different clinical meanings.

Electronic Health Records / EHRs - The source material for this study, consisting of digitized, unstructured clinical narratives (i.e., discharge summaries, prescriptions, and progress notes) from which medical terms are extracted.

Failure Link Chain Traversal - The iterative process in the standard Aho-Corasick algorithm where the search follows pointers back to previous states during a mismatch; in this study, it is the specific lapse targeted for elimination via precomputed skip tables.

Noise - Refers to any unwanted mess or irregularities in the input data that interfere with the accurate extraction and recognition of medical terms.

Transformer-based Optical Character Recognition / TrOCR - The process of converting images of typed, handwritten, or printed text into machine-encoded



text. OCR serves as the primary data-entry layer that translates digitized images of doctors' prescriptions into raw string sequences

Priority-Weighted Scoring - The ranking mechanism developed in this study to resolve overlaps or ambiguities when a single segment of text matches multiple potential medical terms in the dictionary.

Skip Table - A precomputed transition matrix created during the build phase of the enhanced algorithm to provide a constant-time, direct destination for any state-symbol combination, bypassing the need for traditional failure links.

Tiered Hot/Cold Transition Storage - The memory optimization strategy that classifies trie states based on frequency of use, employing flat arrays for "hot" (frequent) states and hash maps for "cold" (infrequent) states to reduce the overall memory footprint of the medical dictionary.

Uncoalesced Memory Access - The inefficient hardware behavior identified in the problem statement where threads access scattered, non-contiguous memory locations; this study seeks to mitigate this by reorganizing trie nodes into a Breadth-First Search (BFS) contiguous layout.

CSV/RRF - A structured file format used in this study to store and organize the medical dictionary, categorizing clinical entities into five fields: Drug, Dosage, Frequency, Abbreviation, and Treatment, which the enhanced Aho-Corasick algorithm uses to map extracted text to their respective medical classifications.

REST API - Representational State Transfer Application Programming Interface; a communication architecture used in this study to connect the enhanced Aho-Corasick backend search engine to the front-end web interface, enabling the transmission of extracted and validated medical terms to the user display.

HTML/CSS - HyperText Markup Language and Cascading Style Sheets; the front-end technologies used in this study to build the web-based interface where



users upload prescription images and view the extracted medical terms with color-coded annotations and a structured results table.

OCR (Optical Character Recognition) - The process of converting images of handwritten or printed text into machine-readable strings. In this study, it is implemented through TrOCR, which digitizes handwritten doctors' prescriptions into raw text before medical term extraction is performed.



Chapter TWO

REVIEW OF RELATED LITERATURE AND STUDIES

This chapter presents a comprehensive review of foreign and local literature and studies relevant to OCR, string-matching algorithms, and medical term extraction. Highlighting the ongoing reliance on deterministic string-matching algorithms like Aho-Corasick and its limitations in handling linguistic noise, contextual ambiguity, and structural inconsistencies in Philippine clinical notes. This chapter justifies the necessity of an enhanced, context-aware, and approximate matching framework, providing the theoretical foundation for improving medical data extraction.

2.1 Related Literature

NLP & RULE-BASED EXTRACTION IN HEALTHCARE

The article by Bazoge et al. (2023), entitled “Applying Natural Language Processing to Textual Data From Clinical Data Warehouses: Systematic Review,” discussed how Natural Language Processing (NLP) based information extraction techniques, particularly pattern matching, transform unstructured clinical text to structured data. The study focuses on examining 194 peer-reviewed articles that used the NLP method in clinical data warehouses (CDW), a database where hospitals store patient information, and it proves that the common focus among these studies was information extraction using clinical NLP. Upon examining them, they also noticed that most of them used the traditional rule-based method, like regular expression or pattern matching, for extracting medical terminologies other than using advanced approaches like deep learning. This proves that classical NLP techniques like string matching are still practical and most widely used by medical fields. These findings support the relevance of the present study, which is the enhancement of the Aho-Corasick algorithm, a string matching algorithm for effectively extracting medical terms from unstructured clinical text. However, while string matching methods remain widely used, identifying their



specific limitations in medical term extraction is equally important in justifying the need for the present study's proposed enhancements.

Moreover, according to Ngo and Koopman (2024), entitled "From Free-Text Drug Labels to Structured Medication Terminology with BERT and GPT," the extraction of medical terms from free-text drug labels using different Named Entity Recognition (NER) techniques, including the Aho-Corasick algorithm. The study focuses on comparing three approaches: the rule-based (dictionary matching), machine learning, and LLMs/transformers (BERT and GPT), and they identified that the Trie-based Aho-Corasick algorithm, which is under dictionary matching, is one of the non-learning approaches that is used as the baseline to compare to more advanced transformer and GPT-based systems. Additionally, in the dictionary-based approach, a vocabulary of medical terms is first created from standard resources such as SNOMED CT, and then the NER task is considered as a string-matching problem, where all matching terms from the dictionary are searched within the input text. This is when the Aho-Corasick algorithm is used as the core engine since it can perform linear-time multi-pattern matching, making it faster even if there are many terms to search, making it still practical to use. But based on the given result, the accuracy of the transformer-based models is higher than the dictionary-based method since it depends on the size and the quality of the collected vocabulary, and it cannot identify a term's different roles inside a given text. These findings are important in the present study because they prove that even though the Aho-Corasick algorithm is a standard tool that can be used for dictionary-based medical term extraction, it also has a limitation that must be further improved for more approximate matching.

Similarly, the research conducted by Navarro et al. (2023) examines how NLP is used in Named Entity Recognition (NER) and its relation to the extraction of clinical free text. They search studies from 2010 onwards from databases such as PubMed, Scopus, ACL, and ACM, and they analyze what method these studies used for clinical NLP. As a result of their research, they found out that machine learning methods are the most



dominant, with transformer-based models increasingly taking the lead for contextual understanding. However, they also found that only a few studies had been translated into real clinical practice, which proves that accurately capturing context remains a problem in clinical NLP, particularly because the same term can carry different clinical meanings depending on its surrounding text. This study supports the present study by establishing the importance of context-aware validation layers integrated into the enhanced Aho-Corasick algorithm to distinguish ambiguous medical terms from valid ones.

Additionally, the study conducted by Gholipour et al. (2023), entitled "Extracting Cancer Concepts from Clinical Notes Using Natural Language Processing: A Systematic Review," provides a broad examination of the NLP methods used to automatically identify cancer-related concepts from free-text clinical documentation. The study searched PubMed, Scopus, Web of Science, and Embase for relevant articles and analyzed the algorithms used for clinical concept extraction in oncology settings. Upon reviewing the gathered studies, they found that rule-based algorithms were the most frequently used method for extracting clinical concepts, with researchers consistently favoring pattern matching and dictionary-based approaches over machine learning counterparts due to their higher accuracy and sensitivity in identifying domain-specific medical terminologies. Their conclusion further recommends that future research continue to prioritize and optimize rule-based approaches when dealing with structured concept recognition in clinical text. This study is highly relevant to the present research because it reinforces the validity of using rule-based and dictionary-based pattern matching, which is the foundational mechanism of the Aho-Corasick algorithm, as the primary extraction strategy for clinical medical terms. Furthermore, its findings on the inherent limitations of purely syntactic matching when terms appear in variant or abbreviated forms directly justify the need for the approximate and context-aware enhancements proposed in the present study.

Furthermore, the research conducted by Durango et al. (2023), entitled "Named Entity Recognition in Electronic Health Records: A Methodological Review,"



systematically traces the evolution of Named Entity Recognition (NER) methods applied to Electronic Health Records (EHR) from 2011 to 2022. The study classifies NER systems by their underlying classification models and tagging architectures and analyzes the progression from rule-based approaches to deep learning methods such as BiLSTM-CRF and BERT-based transformers. Their review identifies that while deep learning methods have increasingly dominated recent NER implementations due to superior F1-scores in benchmark tasks, a persistent challenge across all approaches is the accurate handling of domain-specific abbreviations, acronyms, and irregular clinical language, which continues to degrade performance in real-world EHR environments. They further note that the dominance of deep learning methods in NER has not yet translated into consistent gains in low-resource clinical settings where labeled training data is scarce. This study is relevant to the present research as it establishes the technical landscape in which the Aho-Corasick algorithm operates, confirming that dictionary-based and rule-based methods remain a necessary and practical baseline, particularly in settings like Philippine clinical notes, where the domain-specific irregularities identified by Durango et al. are especially pronounced and where deep learning methods require more training data than is readily available.

PERFORMANCE OPTIMIZATION & MODERN ALGORITHMIC CHALLENGES

The study of Celebi and Yavanoğlu (2023) entitled “Accelerating Pattern Matching Using a Novel Multi-Pattern-Matching Algorithm on GPU” discusses the major inefficiencies of traditional multi-pattern matching algorithms like Aho-Corasick in terms of memory and processing speed, especially when used in large-scale and high-performance applications. The study proposed GPU-based algorithms, they used parallel processing to handle multiple characters at once, and they evaluated it against Aho-Corasick and Wu-Manber. They used 500 patterns as a dataset. As a result, they get 425 times memory usage compared to AC and 3.5 times higher throughput and identify that the main bottleneck of AC is uncoalesced memory access. These findings show that algorithm logic is not only the problem; memory structure and layout must also be taken



into consideration. The study further established that compressing the transition table to reduce memory requirements and improve cache utilization is one of the most critical ongoing research directions in Aho-Corasick optimization.

Moreover, the study by Redwan et al. (2024), entitled "Optimizing Big Data Pattern Search: Advanced String Manipulation Techniques in C++," conducts a systematic performance comparison of five major string pattern matching algorithms, namely Knuth-Morris-Pratt (KMP), Aho-Corasick, Finite Automata, Boyer-Moore, and Rabin-Karp, under big data conditions using C++ optimization techniques. The study evaluates each algorithm's strengths and limitations as dataset size scales upward and applies low-level C++ optimizations, such as disabling stream synchronization and minimizing memory overhead, to all implementations. Their results confirm that Aho-Corasick is the dominant algorithm for large datasets requiring simultaneous matching of multiple patterns, significantly outperforming single-pattern methods like KMP in multi-pattern environments. However, they also note that even the optimized Aho-Corasick implementation faces degraded performance as the transition table grows in size, reinforcing memory structure as a persistent architectural bottleneck. This study is significant to the present research because it provides an empirical, multi-algorithm benchmark that contextualizes the choice of Aho-Corasick as the core engine for the proposed medical term extraction system. Furthermore, its findings on performance degradation under large pattern sets directly justify the memory optimization strategies; particularly the Tiered Hot/Cold Transition Storage approach, incorporated into the enhanced Aho-Corasick framework of the present study.

Furthermore, a study by Ashok et al. (2024), titled "Advancements in String-Searching Algorithms: A Novel Approach," evaluates the advancement of dictionary-based search mechanisms, specifically highlighting the limitations of the baseline Aho-Corasick algorithm in handling real-world data variations. Their research emphasizes that while AC remains the most efficient for high-speed and multi-pattern matching, it requires significant modifications to remain robust against the character-level



noise and structural misalignments often found in automated data extraction. By proposing a hybridized search model that utilizes advanced indexing and approximate matching logic during the trie-traversal phase, the study proves that it is possible to maintain computational speed while increasing the flexibility of the search engine. This foreign research is particularly significant to the present study as it provides a modern technical justification for the integration of a context-aware approximate matching layer into the Aho-Corasick baseline, directly supporting the study's goal of robustly extracting medical terms from noisy and handwritten real-world inputs.

Finally, the study by Khancome et al. (2026), entitled "Multiple String Pattern Matching Algorithm Using Multi-Character Inverted Lists," introduces MMIVL, a high-performance multi-pattern string matching algorithm built on a novel multi-character inverted list data structure (m-CIVL) that directly addresses known computational trade-offs in existing approaches such as Aho-Corasick. The proposed m-CIVL framework integrates positional information, termination semantics, and pattern associations into a single unified structure, enabling direct pattern matching without the need for a separate verification stage, a feature that the Aho-Corasick automaton's failure-link traversal phase typically necessitates. The algorithm achieves preprocessing complexity of $O(|P|/s)$, where s is the segment length, and search complexity of $O(|T| + \text{nocc})$, while also supporting dynamic pattern updates at $O(|p|/s)$ cost, a capability that the static Aho-Corasick trie inherently lacks. Their experimental evaluation demonstrates that MMIVL delivers measurable throughput improvements over baseline multi-pattern algorithms in high-volume text environments. This study is relevant to the present research because it identifies the specific bottlenecks; namely the preprocessing overhead and static structure, of the Aho-Corasick algorithm under dynamic and large-scale conditions. This directly informs the structural modifications in the present study's enhanced AC algorithm, particularly the rationale for tiered storage and context-aware traversal, which similarly seek to reduce redundant state transitions and optimize the algorithm's response to the variable-length and noise-prone patterns found in Philippine clinical notes and medical term dictionaries.



IN THE LOCAL CONTEXT: NLP & DIGITAL HEALTH

In the local context, the study conducted by Robantes et al. (2024) entitled “Enhanced Named Entity Recognition Algorithm for Filipino Cultural and Heritage Texts” explains that standard Named Entity Recognition (NER) models are not enough when dealing with domain-specific and low-resource Philippine texts. The study uses a hybrid approach that combines rule-based pattern matching, domain-specific datasets, and structured ontology to improve performance. Their system integrates the CIDOC-CRM ontology and a pattern-based method to better recognize and classify cultural entities. By using a curated dataset called COHFIE, they were able to improve the identification of entities such as cultural terms, locations, persons, and organizations. The study shows that using structured knowledge and pattern-based techniques is important for handling the unique challenges of Filipino language data, especially when there is limited training data. It also suggests that combining rule-based and machine learning approaches can further improve results in the future. This literature is relevant to the present study because it highlights the importance of pattern-based methods in Philippine NLP applications, supporting the use of the Aho-Corasick algorithm as a foundation for extracting medical terms, especially when enhanced to handle variations and ambiguity in clinical text.

Additionally, in the context of national-scale digital health transformation, Zuniga et al. (2026) in their study titled “Development and Implementation of an Interoperable Local Health Information Exchange Platform for the Attainment of Universal Health Coverage in the Philippines” emphasize the critical need for technical interoperability within the Philippine health information exchange (PHIE). The researchers highlight that a primary challenge in achieving Universal Health Coverage (UHC) is the fragmented and non-standardized nature of clinical data captured across various local health units (LHUs) and primary care providers. Their implementation of a consistent platform reveals that even with a centralized data exchange, the quality of information is often



compromised due to data inconsistencies and the lack of structured reporting at the point of care. This literature is highly relevant to the present research as it establishes the institutional and technical necessity for robust extraction tools; by utilizing an enhanced Aho-Corasick algorithm with context-aware and approximate matching, the present study provides the necessary layer to process the fragmented and non-standardized clinical notes identified by Zuniga et al. effectively. This ensures that even within a framework, the specific medical terms required for the National Health Data Repository can be extracted with high precision despite the inherent noise in localized health records.

Moreover, in the field of localized natural language processing, Montalan et al. (2025), in their study titled “Batayan: A Filipino NLP benchmark for evaluating Large Language Model,” they establish a comprehensive framework for assessing the performance of computational models on Philippine-specific linguistic tasks. The researchers identify that most global NLP benchmarks fail to account for the complexity and code-switching tendencies prevalent in Filipino text, leading to lower accuracy in domain-specific applications. By introducing the Batayan benchmark, they provide a standardized way to measure how well models handle the unique structural and semantic nuances of the Filipino language. This study is highly relevant to the present research as it underscores the technical necessity for context-aware and pattern-based approaches when dealing with low-resource Filipino data. By utilizing an enhanced Aho-Corasick algorithm with a context-aware layer, the present study addresses the ‘morphological complexity’ and linguistic nuances identified by Montalan et al., ensuring that the extraction of medical terms is robust against the specific challenges of the Filipino linguistic landscape.

Furthermore, in the context of the socio-technical challenges within the Philippine healthcare system, De Mesa et al. (2024), in their study entitled “Facing the digital frontier: exploring user acceptance of electronic health records in an urban, rural and remote setting in the Philippines,” it examines the critical factors influencing the adoption of digital health tools across different geographical settings. The researchers identify that



a major barrier to the successful use of Electronic Health Records (EHR) is the technical complexity and lack of localized usability which leads to inconsistent data entry by healthcare practitioners in diverse environments . Their findings highlight that users in rural and remote settings often struggle with systems that are not resilient to the unstructured and informal ways raw clinical data is traditionally recorded in those areas. This study is highly relevant to the present research as it provides a socio-technical justification for a more flexible extraction engine; by enhancing the Aho-Corasick algorithm with approximate matching, the present study offers a solution to the"unstructured input identified by De Mesa et al., ensuring that vital medical terms can still be captured even when technical and environmental constraints lead to non-standardized digital documentation.

Similarly, on the topic of the digital divide within the Philippine healthcare landscape, Caballes (2021), in the study entitled “Acceptability of Information Technology Systems Developed for Distant Philippine Communities among Local Health Providers,” it investigates the factors influencing the adoption of health IT systems in geographically isolated and disadvantaged areas (GIDAs). The research identifies that a critical barrier to system acceptance is the technical mismatch between standardized software requirements and the informal plus narrative-heavy way local health providers document patient encounters. The study concludes that for technology to be sustainable in distant Philippine communities, it must be flexible enough to handle the non-standardized and often noisy data entry styles of practitioners who prioritize patient care over rigid data protocols. This research is highly relevant to the present study as it provides a localized justification for a more resilient extraction framework. By enhancing the AC baseline with an approximate matching layer, the present study directly addresses the technical mismatch identified by Caballes, ensuring that medical terms can be extracted even when digital records are captured in the informal and unstructured formats common in distant Philippine healthcare settings and amongst legacy healthcare practitioners.



2.2 Related Studies

EVALUATION ON STRING-MATCHING FRAMEWORKS & DEEP LEARNING LIMITATION IN MEDICAL INFORMATION RETRIEVAL

The study of Alfattni et al. (2021) entitled “Extracting Drug Names and Associated Attributes From Discharge Summaries: Text Mining Study” examined how feasible deep learning-based NLP is to extract drug names and related medication attributes from free-text clinical discharge summaries. The study used a DrugEx system, which is based on BiLSTM-CRF architecture, a deep learning model for sequence tagging. They also use different word embedding strategies, such as pretrained word embeddings, which convert words into vectors to determine the words with similar meaning based on the distance of the vectors to each other. They also use character embeddings to check spelling variations and unknown words. They evaluated this strategy using the MIMIC-III dataset from n2c2 2018 shared tasks, and the datasets came from annotated clinical documents. Their result showed that combining pretrained word embeddings with character embeddings achieved higher performance in drug name extraction, but it struggled in terms of contextually ambiguous terms. This study is relevant to the present research because it confirms that even advanced deep learning systems face limitations when medical terms appear in ambiguous contexts, directly highlighting the need for a context-aware matching mechanism such as the one proposed in the enhanced Aho-Corasick algorithm.

Conversely, the study by Karthikeyan et al. (2021), entitled "An OCR Post-Correction Approach Using Deep Learning for Processing Medical Reports," examines the challenges introduced by Optical Character Recognition (OCR) errors in clinical documentation and proposes a post-correction framework to address them. The study evaluates OCR-generated output from NHS medical reports and a secondary biomedical dataset, finding that OCR systems frequently fail to accurately transcribe scanned or handwritten clinical documents when text is skewed, obscured, or otherwise degraded, leading to elevated Word Error Rates (WER) and Character Error Rates (CER)



in the resulting digital text. They developed a deep learning-based post-correction pipeline using the RoBERTa language model, which significantly reduced both WER and CER averages on the test datasets. However, their findings underscore that OCR post-correction alone is insufficient for ensuring accurate term identification without a downstream matching layer capable of tolerating residual character-level noise. This study is directly relevant to the present research because it confirms that even after deep learning-based correction, OCR-processed medical text retains character-level errors that a string-matching system must handle. This validates the necessity of the approximate matching layer in the enhanced Aho-Corasick algorithm, which is specifically designed to tolerate these residual OCR-induced noise patterns and map imperfect text strings to their correct medical dictionary entries.

Similarly, the research conducted by Mahajan et al. (2023), entitled “Overview of the 2022 n2c2 Shared Task on Contextualized Medication Event Extraction in Clinical Notes,” discusses the challenges in extracting medical information from clinical text. This study highlights how lexical variability such as wrong spellings and abbreviations and lack of context in clinical notes affects the accuracy of automated extraction systems. The study examines 32 teams from 19 countries who submitted 211 systems across three subtasks, such as medication extraction (NER), event classification, and context classification, using real clinical notes as a dataset. As a result of the study, the medication name extraction receives higher performance, while event and context classification receives lower performance due to some events not being clearly stated, requiring reliance on contextual clues for proper interpretation, and sometimes multiple or overlapping events are mentioned for a single medication. This study demonstrates that while NLP has matured enough for medication extraction, it still lacks trust when it comes to understanding the context in which medical terms are used in clinical notes. These findings clearly support the present study by making medical extraction and approximate matching possible using the enhanced Aho-Corasick algorithm to make it more flexible in handling clinical text.



Moreover, the study of Liu et al. (2024), entitled "Evaluating Medical Entity Recognition in Health Care: Entity Model Quantitative Study," conducted a broad quantitative evaluation of Named Entity Recognition (NER) models applied to unstructured medical texts, focusing on how well general-purpose models perform on domain-specific clinical data. The study evaluated multiple NER architectures across disease, treatment, and condition entity types and found that model performance is highly sensitive to the domain specificity of the training data; models trained on general-purpose corpora performed substantially worse when applied to specialized medical terminology, emphasizing the necessity of domain-targeted model optimization and fine-tuning. Their analysis also reveals that even high-performing NER models struggle with accurately recognizing entities in the presence of irregular clinical phrasing, non-standardized terminology, and co-occurring clinical abbreviations. They conclude that the key bottleneck in medical entity recognition is not model complexity but rather the alignment between the model's knowledge base and the specific terminological landscape of the clinical domain being processed. This study is directly relevant to the present research because it validates the importance of a domain-specific, dictionary-anchored approach, the core strength of the Aho-Corasick algorithm, over generic learning models for accurate medical term extraction, and further reinforces the necessity of enhancing that dictionary-based core with approximate and context-aware layers to bridge the gap when clinical terminology in Philippine patient notes deviates from standardized forms.

ALGORITHMIC EFFICIENCY & STRUCTURAL OPTIMIZATION

The study of Fayeze et al. (2024) entitled "Enhanced Aho-Corasick Algorithm for Network Intrusion Detection Systems" examined the memory constraints of the traditional Aho-Corasick algorithm in relation to large-scale pattern matching in Network Intrusion Detection Systems (NIDS). They developed an improved version of the Aho-Corasick algorithm, which is the parallel failure-less compressed hashed variation where it uses parallel processing for handling multiple tasks at once, removes failure transitions, and applies compression and hashing techniques to be able to reduce memory



consumption and achieve accurate pattern matching even with large datasets. Their results showed that the compressed hashed variation reduced memory usage to only the square root of what the original algorithm required, though this came at the cost of a 4 to 5 times slower execution time due to the overhead of hash calculations. This study is directly relevant to the present study as it proves that the standard Aho-Corasick algorithm has a memory inefficiency that needs to be addressed, supporting the motivation behind the Tiered Hot/Cold Transition Storage optimization proposed in this study.

Additionally, the research conducted by Kate (2021), entitled "Clinical Term Normalization Using Learned Edit Patterns and Subconcept Matching," presents a compelling and technically direct justification for approximate matching in clinical NLP. It addresses the challenge of mapping clinical mentions from free-text patient notes to their standardized forms in clinical terminologies such as the Unified Medical Language System (UMLS). Crucially, the study reveals that in a standardized test corpus of 100 discharge summaries, only 62.37% of clinical terms were matched exactly to their corresponding UMLS entries, meaning that nearly 40% of clinical mentions required some form of approximate or variant-aware matching to be correctly normalized. The proposed system addresses this gap by learning edit patterns, character-level transformation rules derived from training data which allow the matching engine to account for morphological and orthographic variations between how clinical terms are written in practice and how they appear in controlled vocabularies. Their results demonstrate that systems augmented with edit-distance-based approximate matching consistently outperform strict dictionary matching baselines across multiple clinical concept normalization benchmarks. This study is directly significant to the present research because it provides measurable proof that exact string matching is fundamentally insufficient for real-world clinical term identification, quantifying the exact gap that the approximate matching layer of the enhanced Aho-Corasick algorithm is designed to close. The edit-pattern learning methodology described by Kate further



supports the theoretical basis of integrating edit-distance-informed approximate matching into the Aho-Corasick trie traversal phase, as proposed in the present study.

Furthermore, Gor and Bhensdadia (2025) presents a "Comparative Analysis of Classical String Matching Algorithms," which explicitly evaluates the performance of the Aho-Corasick algorithm against other deterministic methods like KMP and Boyer-Moore. The research highlights that while the AC algorithm is the leading standard for multi-pattern speed, its traditional all-or-nothing matching logic is inherently sensitive to inconsistencies due to data noise. The authors suggest that in environments where data sequences are imperfect, such as digitized or OCR-processed text, classical algorithms must be augmented with more flexible logic to remain effective. This foreign study, while international in scope, is applied to the local context by validating the choice of AC as a high-speed core engine for medical dictionaries. It provides a technical precedent for modifying the AC trie structure to allow for approximate matches, ensuring that the algorithm does not return null results when encountering minor transcription errors. This research is crucial for the present study, as it justifies the transformation of a rigid deterministic algorithm into a robust and noise-aware extraction framework.

THE PHILIPPINE HEALTHCARE LANDSCAPE: DATA NOISE AND LOCALIZED CHALLENGES

A methodical analysis by Tinam-isan and Naga (2024), titled “Exploring the Landscape of Health Information Systems in the Philippines,” underscores the persistent issues of compromised data quality and the lack of information organization in the local healthcare sector. The study argues that the fragmented nature of clinical documentation in the Philippines leads to significant difficulties in structuring patient data for automated systems. These challenges are worsened by the lack of standardized terminology across different health facilities, which mirrors the ambiguity found in handwritten patient notes and prescriptions. This literature is highly relevant as it highlights the information gap that exists between raw clinical inputs and structured digital records. It supports the



integration of a context-aware validation layer in the present study, as it is required to resolve the inconsistencies and non-standardized abbreviations common in the Philippine medical landscape. Ultimately, the study reinforces the goal of creating a more reliable tool for clinical information practice that can function despite the systematic lack of data uniformity.

In the local context, the study conducted by Espinas et al. (2025) entitled “Real-Time CNN-Based Generic Medicine Name Classification for Prescription Recognition” discusses that despite the use of advanced machine learning models (CNN), it still has a limitation in recognizing medical names, especially with similar characters. In this study they developed a real-time web application to classify 20 generic medicine names using the CNN model for handwritten prescription recognition. As a result, the system is generally reliable, but it has lower accuracy in terms of medical terms such as “chlorpromazine” and “Hydroxyzine” due to the similarity of characters that need to have more training data. This study is therefore relevant to the present research because it demonstrates that even machine learning-based approaches face accuracy limitations when recognizing medicine names with similar character patterns, which is a problem that the present study's approximate matching and context-aware layers in the enhanced Aho-Corasick algorithm are specifically designed to address at the string-matching level.

Similarly, the research conducted by Benitez et al. (2024) entitled “Transcribing Handwritten Medical Prescriptions using Convolutional Neural Network AlexNet Architecture and Canny Edge Detection” discusses the improvement of handwritten prescription recognition using advanced models. The study uses the CNN AlexNet architecture and Canny edge detection for preprocessing, which includes RGB to grayscale, binarization, edge detection, and inversion compared to the previous CRNN model. As a result, it gets 90.76% accuracy, making preprocessing and testing time faster. However, it does not fully resolve character-level errors, especially on drug names that are visually similar. This study is directly relevant to the present research because it confirms from a Philippine institutional standpoint that even after successful prescription



image recognition, extracted drug name text still contains character-level errors due to handwriting variability, which is precisely the kind of noisy clinical text that the enhanced Aho-Corasick algorithm's approximate matching layer is designed to handle flexibly.

Additionally, a study conducted by Pino and Mendoza (2025), titled “Optical Recognition of the Philippines' Ancient Text: A Deep Learning Approach,” highlights the inherent difficulty of digitizing handwritten text in a local context. While the study primarily focuses on ancient scripts, its findings on the stochastic nature of handwriting, specifically the variations in stroke pressure, ligatures, and character-level noise, are directly applicable to modern handwritten prescriptions. The authors demonstrate that these real-world noises frequently result in fragmented or discontinuous textual output during the OCR phase. This research is highly significant to the present study as it establishes the probabilistic nature of the input data; it suggests that in the local context, where handwriting styles lack rigid standardization, the resulting digital strings are inherently noisy. By addressing this fragmented output problem through approximate matching, the present study directly builds upon their findings, transforming unstable OCR raw data into a robust and verified medical terminology extraction framework, as well as building upon the foundational challenges of localized digitizing identified by Pino and Mendoza.

Moreover, a study by Falcon et al. (2024) titled “Exploring Filipino Medical Students’ Attitudes and Perceptions of Artificial Intelligence in Medical Education: A Mixed-Methods Study” highlights a critical socio-technical gap in the local healthcare landscape regarding AI-driven data processing. While the study primarily focuses on the readiness of the local medical community, it reveals a significant concern among Filipino practitioners: the tendency of current AI systems to produce unreliable information when processing clinical data that lacks a localized context. The participants emphasized that for AI to be truly effective in the Philippines, it must cater specifically to the community-oriented medical orientation of the country, where semantic and technical



errors in data input remain a barrier (Falcon et al., 2024). This research is significant to the present study as it justifies the inclusion of a priority-weighted matching layer; by weighing terms based on their clinical relevance within the Filipino medical context, the enhanced Aho-Corasick algorithm addresses the unreliability and semantic errors identified by local medical students, ensuring that the extraction process is not just accurate but also contextually aligned with local clinical priorities.

In a recent implementation study conducted by Elepaño et al. (2025), in their work titled “Implementing Electronic Health Records in Philippine Primary Care Settings: Mixed-Methods Pilot Study,” they examined the deployment of digital systems across urban (Quezon City), rural (Bataan), and remote (Sorsogon) sites. The researchers found that as systems transitioned from open-source to proprietary platforms, there was a notable decline in acceptability due to technical disparities and contextual factors that varied by location . Their findings emphasize that digital health tools in the Philippines must be adaptable to the specific needs and capacities of each setting to overcome data inconsistencies. This research is directly relevant to the present study as it confirms the presence of contextual and technical noise within the local healthcare infrastructure. By applying a context-aware and priority-weighted approach to the Aho-Corasick algorithm, this research provides a technical solution to the disparities identified by Amit et al., ensuring medical term extraction remains robust even when data is collected from varying and unstandardized local primary care contexts.

2.3 Research Synthesis

The review of related literature and studies shows that there is an urgent need for a more robust and context-sensitive approach to extract medical terms from unstructured clinical text. From a clinical informatics perspective, Bazoge et al. (2023), Navarro et al. (2023), and Mahajan et al. (2023) show the consistency of rule-based and dictionary-matching methods like Aho-Corasick being widely used in clinical Natural Language Processing (NLP) pipelines. This is further reinforced by Gholipour et al. (2023), whose systematic review of NLP methods for cancer concept extraction found that rule-based



and dictionary-based approaches were the most frequently favored strategies due to their higher accuracy and sensitivity in identifying domain-specific medical terminologies, directly validating the foundational mechanism of the Aho-Corasick algorithm as a primary extraction strategy for clinical terms. However, these methods face a major limitation in handling spelling variations, abbreviations, and contextual ambiguity, which are inevitable in real-world medical documentation. Kate (2021) quantifies this insufficiency directly, finding that in a standardized corpus of 100 discharge summaries, only 62.37% of clinical terms were matched exactly to their UMLS entries, meaning nearly 40% of clinical mentions required approximate or variant-aware matching for correct normalization, providing measurable proof that strict dictionary matching alone cannot serve as a complete solution. Additionally, the study of Ngo and Koopman (2024) shows that even transformer-based models, which have higher accuracy than rule-based models like Aho-Corasick, still serve as a practical and efficient baseline especially when computational resources are limited. El-Allaly et al. (2021) reinforce this by showing that even deep learning systems face a bottleneck in identifying contextually ambiguous medical terms, and Durango et al. (2023), whose systematic review of NER methods in EHRs from 2011 to 2022 found that a persistent challenge across all approaches remains the accurate handling of domain-specific abbreviations, acronyms, and irregular clinical language, further confirm that context-awareness is a challenge that spans both traditional and modern approaches. Liu et al. (2024) similarly found that even high-performing NER models struggle with irregular clinical phrasing and non-standardized terminology, concluding that the key bottleneck in medical entity recognition is not model complexity but the alignment between the model's knowledge base and the specific terminological landscape of the target clinical domain, which validates the importance of a domain-specific, dictionary-anchored approach augmented with context-aware layers.

On the algorithmic and systems side, Çelebi and Yavanoğlu (2023), Feyeze et al. (2024), and Gor and Bhensdadia (2025) identify three major weaknesses of the standard Aho-Corasick algorithm: repeated failure link traversals, which slow down search performance; scattered memory allocation, which causes cache inefficiency; and full-alphabet transition storage per state, which results in significant memory waste. According to Kanda et al. (2023) and Samyal et al. (2025), these issues are not only theoretical but also have a direct impact on real-world performance, especially in large-scale



pattern matching systems. Redwan et al. (2024) provide empirical confirmation through a systematic multi-algorithm benchmark, demonstrating that while Aho-Corasick significantly outperforms single-pattern methods in multi-pattern environments, even optimized implementations face degraded performance as the transition table grows, reinforcing memory structure as a persistent architectural bottleneck. Al Sohan et al. (2024) further identify that the preprocessing overhead and static trie structure of Aho-Corasick are specific bottlenecks under dynamic and large-scale conditions, directly informing the structural modifications proposed in the present study. Meanwhile, Ashok et al. (2024) show that it is possible to augment the Aho-Corasick trie using approximate matching while preserving computational speed, which supports the direction of the current study.

In the local context, the studies of Tinam-isan and Naga (2024), De Mesa et al. (2024), Caballes (2021), and Elepaño et al. (2025) are unified in a single conclusion: clinical documentation in the Philippines is fragmented, not standardized, and usually handwritten or informally formatted, making it difficult to process using rigid automated systems. Additionally, Benitez et al. (2024) and Espinas et al. (2025) show that even when using Optical Character Recognition (OCR) for handwritten prescriptions, character-level errors remain a problem, especially in drug names that existing systems cannot correct, and Karthikeyan et al. (2021) further confirm that even after applying deep learning-based OCR post-correction, medical text retains residual character-level errors that a downstream matching system must still tolerate, validating the necessity of the approximate matching layer for any system processing OCR-derived clinical text. The studies of Robantes et al. (2024), Montalan et al. (2025), and Pino and Mendoza (2025) also show that pattern-based methods with domain-specific context are not only viable but necessary when processing low-resource and noisy Filipino language data. Concurrently, Zuniga et al. (2026) and Falcon et al. (2024) contextualize the problem at a national scale, highlighting that fragmented clinical data and the lack of localized intelligence in automated systems are active barriers to healthcare interoperability and patient safety in the Philippines.

Overall, the reviewed literature and studies reveal a research gap that the present study directly addresses: while the Aho-Corasick algorithm is widely recognized as the standard engine for dictionary-based term extraction, and while the reviewed studies



collectively confirm both the necessity of rule-based extraction and the insufficiency of exact matching in real-world clinical settings, no existing work has simultaneously resolved its three core hardware-level inefficiencies, repeated failure link traversal, scattered memory allocation, and full-alphabet transition storage while also integrating context-aware and priority-weighted approximate matching tailored to the noisy, abbreviation-heavy, and locally variable nature of clinical notes. Prior algorithmic improvements either focus on memory optimization at the cost of execution speed, or add context-awareness as a disconnected post-processing step rather than embedding it within the search mechanism itself. The present study bridges this gap by proposing a unified enhanced Aho-Corasick framework that addresses structural inefficiencies through a precomputed Skip Table, Two-Pass BFS memory layout, and Tiered Hot/Cold Transition Storage, while simultaneously supporting approximate and context-aware matching that offers a practical, efficient, and locally grounded solution for robust medical term extraction from handwritten patient prescriptions.



Chapter THREE METHODOLOGY

This chapter outlines the systematic approach used to conceptualize and design the enhanced Aho-Corasick algorithm. It details the research design and the theoretical logic behind the proposed modifications to the baseline string-matching process. The discussion focuses on the formulation of the algorithmic blueprint, the structural design of the state-transition machines, and the criteria for establishing a robust experimental environment. By detailing these foundational stages, this chapter provides the theoretical framework necessary to ensure the enhancement and logical validity of the proposed system's design.

3.1 Research Design

The study utilizes an experimental research design, a systematic approach that manipulates specific independent variables to observe their effects on a dependent variable within a controlled environment. By definition, this methodology focuses on establishing cause-and-effect relationships through consistent testing, objective measurement, and the isolation of variables. This design is particularly suited for this research as it allows for a direct comparison between the traditional Aho-Corasick algorithm and the proposed enhanced version. By controlling the input data, specifically noisy medical documentation (example, prescription notes), the study can measure how context-aware and priority-weighted logic impacts string-matching accuracy. Ultimately, this experimental framework ensures that the performance of the proposed algorithm is validated through statistical evidence rather than theoretical assumptions alone.

The research process is structured into three sequential stages, beginning with the OCR Integration and Text Acquisition, which focuses on converting physical or digital medical documents into machine-readable text using Optical Character Recognition technology to establish the raw dataset. This is followed by the Deployment of the Enhanced AC Algorithm, where the proposed system is systematically applied to the



acquired text to identify and extract clinical terms using its priority-weighted and context-aware logic. Finally, the process concludes with the Comparative Analysis and Validation, involving an evaluation where the enhanced algorithm's performance metrics are measured against the traditional AC baseline to ensure precision, reliability, and statistical significance.

By adopting this experimental research design, the study aims to evaluate how the inclusion of context-aware and approximate matching logic influences the sensitivity and processing of the system. The overall goal is to determine if this enhancement successfully mitigates the common errors found in medical data processing, such as typos or varied medical nomenclatures; particularly linguistic noise, contextual ambiguity, and structural inconsistencies inherent in clinical notes.

To test the efficacy of the three-stage experimental framework and the specific system requirements, the study utilizes a dataset consisting of authentic handwritten prescriptions sourced from various medical fields, including general medicine, internal medicine, and pediatrics. This focus on self-collected data ensures that the OCR Integration and Text Acquisition stage is challenged by real-world variability in handwriting styles and ink clarity. While the enhanced Aho-Corasick algorithm is designed for robust extraction, the scope of this evaluation is strictly confined to these doctor's prescriptions; consequently, other clinical documents such as discharge summaries or lab results are excluded to maintain a controlled experimental environment. By isolating the dataset to these specific patient notes, the study can more accurately quantify how well the priority-weighted logic resolves the linguistic noise and structural inconsistencies unique to the Philippine medical context.

3.2 Proposed Algorithm and System Architecture

3.2.1 Proposed Algorithm

INPUT

$$P = \{p_1, p_2, \dots, p_n\}$$

$$T = t_1 \dots t_m$$



Σ = alphabet (alphanumeric + clinical symbols)

θ = hot-state threshold

D = abbreviation meaning dictionary

ENHANCED AC BUILD(P)

// Phase 1: standard trie construction

create root q_0

for each pattern p in P :

insert uppercase (p) into trie via goto transitions $\delta(q, a)$

mark terminal state with $O(q) \leftarrow O(q) \cup \{p\}$

// Phase 2 (Objective 2): Two-Pass BFS memory layout

// Pass 1 assign BFS rank to every node

$\text{bfs_order} \leftarrow []$

enqueue(q_0)

while queue not empty:

$v \leftarrow \text{dequeue}()$

$\text{bfs_rank}(v) \leftarrow |\text{bfs_order}|$

$\text{bfs_order.append}(v)$

for each child u of v in sorted char order:

if u not visited: enqueue(u)

// nodes sharing medical prefixes $\rightarrow$ adjacent in memory

// Pass 2 — remap all child pointers to BFS ranks

for each v in bfs_order :

for each (a, child) in $\delta(v)$:

$\delta(v, a) \leftarrow \text{bfs_rank}(\text{child})$ // now contiguous

$\text{nodes} \leftarrow [\text{bfs_order}[i].\text{node} \text{ for } i \text{ in } 0 \dots |\text{bfs_order}|]$

// Phase 3 (Objective 3): Tiered Hot/Cold classification

for each v in nodes:



```
if  $|\delta(v)| \geq \theta$ : // hot state
store(v)  $\leftarrow$  flat_array[ $|\Sigma|$ ] //  $O(1)$  indexed access
else: // cold state
store(v)  $\leftarrow$  hash_map // only actual transitions stored
// avoids full-alphabet allocation for sparse states

// Phase 4: failure links (standard AC BFS on compacted nodes)
queue  $\leftarrow \emptyset$ 
for each  $a \in \Sigma$  with  $\delta(q_0, a)$  defined:
f( $\delta(q_0, a)$ )  $\leftarrow q_0$ 
enqueue( $\delta(q_0, a)$ )
for each  $a \in \Sigma$  with  $\delta(q_0, a)$  undefined:
 $\delta(q_0, a) \leftarrow q_0$  // root self-loop
while queue not empty:
v  $\leftarrow$  dequeue()
for each  $a \in \Sigma$  with  $\delta(v, a)$  defined:
u  $\leftarrow \delta(v, a)$ 
x  $\leftarrow$  f(v)
while  $\delta(x, a)$  undefined and  $x \neq q_0$ :
x  $\leftarrow$  f(x)
if  $\delta(x, a)$  defined: f(u)  $\leftarrow \delta(x, a)$ 
else: f(u)  $\leftarrow q_0$ 
O(u)  $\leftarrow O(u) \cup O(f(u))$  // inherit outputs
enqueue(u)

// Phase 5 (Objective 1): precompute Skip Table
for each state s in nodes (BFS order):
for each  $a \in \Sigma$ :
if  $\delta(s, a)$  defined:
skip[s][a]  $\leftarrow \delta(s, a)$  // direct transition
```



```

else: // resolve failure chain NOW, once, at build time
x ← f(s)
while  $\delta(x, a)$  undefined and  $x \neq q_0$ :
x ← f(x)
if  $\delta(x, a)$  defined: skip[s][a] ←  $\delta(x, a)$ 
else: skip[s][a] ←  $q_0$ 
// failure chain traversal resolved here — never again at search time
return automaton (nodes,  $\Sigma$ , skip,  $q_0$ , F, f, O, store)

```

ENHANCED AC SEARCH(T)

// Text Normalization

```

T ← uppercase(T) // "biogesic" → "BIOGESIC"
T ← remove_punctuation(T) // "B.I.D." → "BID"
T ← normalize_spaces(T) // extra whitespace removed
T ← expand_common_noise(T) // "ltab" → "l tab" //

```

```

Tokenization tokens ← split T by whitespace and delimiters
token_index ← map each character position to its token

```

```

state ←  $q_0$ 
candidates ← [] // collect hits before filtering

```

```

for each symbol a in T:
// Objective 1: single  $O(1)$  lookup — no failure-link loop
state ← skip[state][a]
if  $O(\text{state}) \neq \emptyset$ :
for each pattern p in  $O(\text{state})$ :
candidates.append( {term: p, pos: current_index} )

```

// Context-aware validation



```
validated ← []
for each hit in candidates:
    window ← tokens surrounding hit.pos // ±k tokens
    if hit.term ∈ AMBIGUOUS_TERMS:
        if window ∩ NEGATIVE_CONTEXT ≠ ∅: skip // example: "cold compress"
        if window ∩ POSITIVE_CONTEXT = ∅: skip // no clinical signal
    hit.context_valid ← true
    validated.append(hit)

// Priority-weighted scoring
for each hit in validated:
    score ← 0
    score += length_bonus(hit.term) // longer = more specific
    score += boundary_bonus(hit.pos, T) // word boundary check
    score += known_term_bonus(hit.term) // in high-priority dictionary
    score += context_bonus(hit.context_valid)
    hit.priority_score ← score / max_score // normalise to [0,1]

// Overlap resolution: keep highest-scoring non-overlapping hits
validated.sort by (-priority, -length)
output ← []
for each hit in sorted:
    overlaps ← output hits that share char range
    if overlaps = ∅:
        output.append(hit)
    else: drop hit

// Meaning for abbreviated terms
for each hit in output:
```



```
if hit.term ∈ D:
```

```
    hit.meaning ← D[hit.term] // abbreviation
```

```
else: hit.meaning ← "—" // not an abbreviation → skip
```

```
// Phase 7: Fuzzy Matching
```

```
unmatched_tokens ← tokens in T not covered by output
```

```
for each token in unmatched_tokens:
```

```
    if len(token) < 3: skip
```

```
    best ← closest match to token in P using edit distance
```

```
        if edit_distance(token, best) ≤ 2 and similarity(token, best)
```

```
≥ 0.6 :
```

```
    output.append({
```

```
        term : token,
```

```
        matched : best,
```

```
        meaning : D[best] })
```

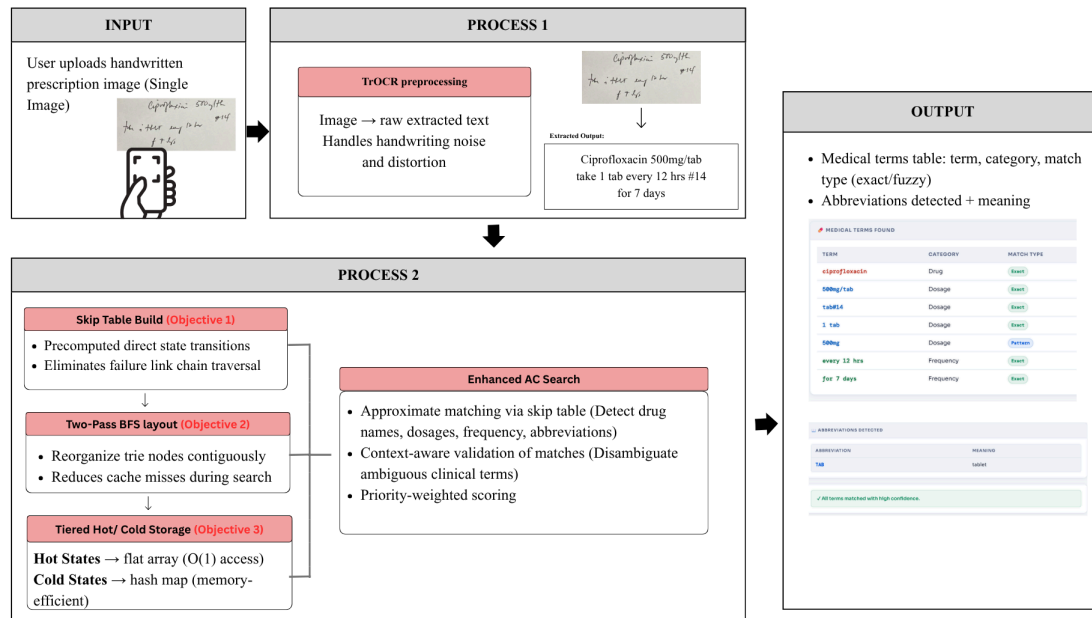
```
report output
```



3.2.2 System Architecture

Figure 3.1

System Process Flow



The system follows a sequential pipeline designed to extract medical terms from handwritten doctors' prescriptions with high accuracy and efficiency. As seen in Figure 3.1, the process begins when the user uploads a prescription image, which is passed through a Transformer-based Optical Character Recognition (TrOCR) module that converts the handwritten content into raw text. This raw text, which often contains noise, spelling irregularities, and informal abbreviations inherent to handwritten clinical documents, is then fed into the core algorithmic pipeline. To address the performance limitations of the standard Aho-Corasick algorithm, three structural enhancements are applied in sequence during the build phase: a precomputed Skip Table that eliminates repeated failure link chain traversals by storing direct state-symbol destination mappings (Objective 1), a Two-Pass Breadth-First Search layout that reorganizes trie nodes into contiguous memory locations to reduce cache misses (Objective 2), and a Tiered Hot/Cold Transition Storage strategy that classifies states as either frequently or



infrequently accessed and allocates memory accordingly, minimizing overall memory waste (Objective 3).

Once the optimized automaton is constructed, the enhanced Aho-Corasick search engine processes the TrOCR-extracted text by performing approximate multi-pattern matching against a medical dictionary of drug names, dosages, and frequency instructions. The search phase integrates a context-aware validation layer that analyzes the surrounding tokens of each candidate match to disambiguate clinically ambiguous terms and a priority-weighted scoring mechanism that resolves overlapping matches by ranking candidates based on their clinical relevance and match confidence. The final output is displayed through the web interface, presenting the extracted text with color-coded annotations, a structured table of identified medical terms categorized by type and match confidence (exact or fuzzy), and a dedicated abbreviations panel that maps shorthand notations to their full clinical meanings.

3.3 System Requirements

To ensure optimal system performance and reliability, this section specifies the essential hardware and software configurations required for the study. These parameters provide the necessary technical foundation to support the enhanced algorithm's primary functions and ensure seamless data processing. The following requirements detail both the minimum and recommended specifications needed to achieve consistent and efficient operational results.

3.3.1 Hardware Requirements

The hardware requirements detail the physical components and processing power needed to efficiently handle OCR tasks and large-scale string-matching operations. These specifications are selected to provide sufficient computational resources for reliable and consistent performance during the study.



Table 3.1

Hardware Specifications for System Implementation

Components	Minimum Requirement
Processor	Intel Core i5 (11th gen)
RAM	8.00 GB
Storage	10 GB available disk space (HDD)
Camera	5 MP smartphone camera or equivalent

For implementing the enhanced Aho-Corasick algorithm, the specified components in table 3.1 are required. To run both the TrOCR model and the Aho-Corasick algorithm smoothly, a computer/laptop with at least an Intel Core i5 (11th Gen) processor is needed to handle processing tasks. A minimum of 8 GB RAM is for basic use, but having 16 GB is recommended for better performance, especially when there are loads of prescription files to process. At least 10 GB of storage is also needed for application files, TrOCR model weights, and other supporting libraries; using an SSD (solid-state drive) instead of an HDD (hard disk drive) is more recommended for faster loading. Other than that, a camera (any smartphone with at least 5 MP) is also needed for capturing clear prescription images that will be used in the OCR process.

3.3.2 Software Requirements

The software requirements specify the operating system, development environments, and specialized libraries necessary to execute the algorithmic logic and manage the data workflow. These tools ensure the compatibility and stability of the system throughout the implementation and testing phases.



Table 3.2

Software Specifications for System Implementation

Components	Minimum Requirement
Operating System	Windows 11 Home (64-bit) / Windows 10 Home (64-bit)
Python	Version 3.8 or higher
Flask	Latest stable release
Pillow (PIL)	Latest stable release
NumPy	Latest stable release
OpenCV (cv2)	Latest stable release
PyTorch	Latest stable release
Hugging Face Transformers	Latest stable release
Visual Studio	2019 or 2022
Web Browser	Any modern browser
Internet Connection	Stable broadband connection

The software requirements specified in table 3.2 have been carefully chosen for the stability and compatibility of the system development and deployment. Microsoft Windows 11 Home (64-bit) serves as the main operating system because of its security and stability, using Windows 10 Home (64-bit) can also be used as an alternative. Python 3.8 or a higher version serves as the main programming language for the backend logic and for utilizing the needed libraries. As for the packages, Flask is used for the web interface and REST API for routing, while Pillow (PIL) and NumPy are used to handle images and numerical preprocessing. OpenCV is used for image processing; it converts images into grayscale, and it applies thresholding, dilation, and contour detection to prepare the prescription images for OCR. PyTorch serves as the deep learning engine for the TrOCR model, and Hugging Face Transformers is used to load the



TrOCR processor and vision encoder decoder model to be able to extract the text from the handwritten prescription. Visual Studio 2019 or 2022 is used for the development and integration of the system. A web browser is also needed for accessing the system, and lastly, a stable internet connection is required for the initial setup of the TrOCR model and other required resources.

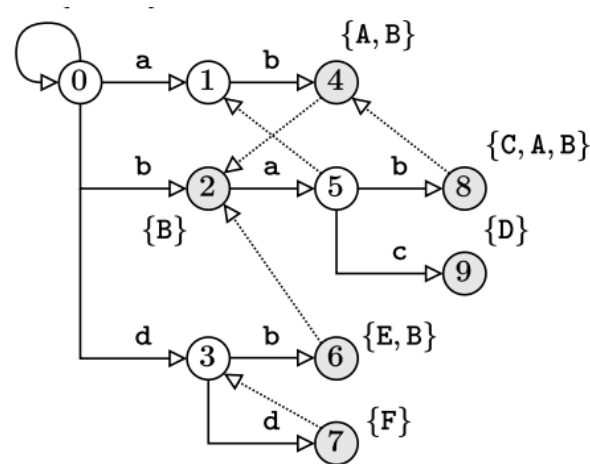
3.4 Theories, Methods, and Tools

3.4.1 Theories

This section presents the theoretical foundations that serve as the theoretical basis for each architectural enhancement applied to the Aho-Corasick algorithm in this study. Each theory is directly mapped to a specific structural modification introduced in this study to address the identified performance and accuracy limitations of the classical algorithm.

Figure 3.2

Aho-Corasick Automaton from patterns {ab, b, bab, bac, db, dd}. Adapted from Kanda et al. (2023)



(a) AC automaton

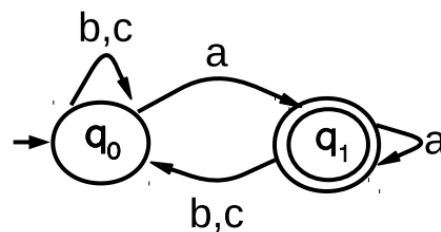
Aho-Corasick Algorithm. Figure 3.2 shows the Aho-Corasick automaton structure showing goto transitions and failure links. Aho-Corasick is a multi-pattern string matching algorithm that builds a trie structure from the set of



patterns, and it adds failure links to ensure that the search can continue without repeating characters in the input text. The theory of this algorithm is based on a deterministic finite automaton (DFA), where each state corresponds to one prefix of one or more patterns and the failure links serve as the fallback when the current character has no transition to go through. The algorithm gives a linear time complexity of $O(|T| + \sum |p_i| + \text{output})$, where T is the length of the input text, $\sum |p_i|$ is the overall length of all the patterns, and output is the count of matched patterns, making it suitable for quickly searching for multiple terms at once.

Figure 3.3

DFA Representation of State Transitions. Adapted from Chakraborty et al. (2023)

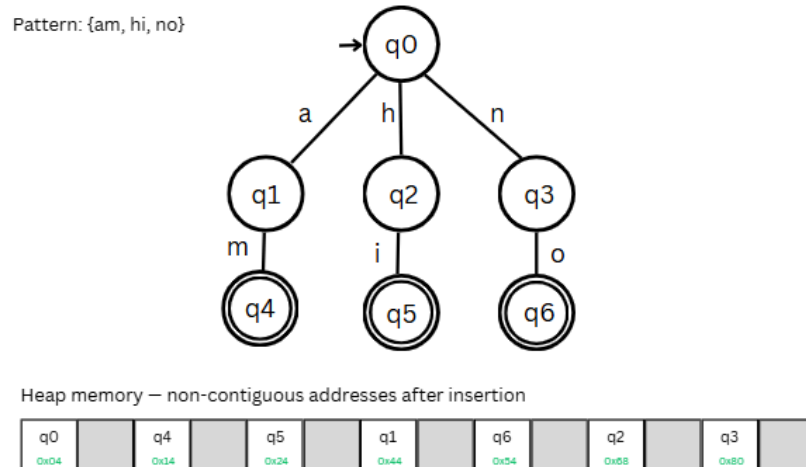


Deterministic Finite Automaton (DFA). Figure 3.3. shows deterministic finite automaton transition structure. It is one of the computational models that consists of a finite set of states, an alphabet, a transition function, a start state, and a set of accept states. Traditional Aho–Corasick (DFA-based) pattern matching uses many states and transitions and it grows as the pattern increases leading to high memory usage and slower processing since each character is handled individually. It also implements an incomplete transition function where undefined transitions are resolved repeatedly at search time through failure link traversal. The enhanced AC automaton in this study transforms this into a complete transition function by using the precomputed skip table during the build time, ensuring that every state symbol has a direct and immediately accessible destination state.



Figure 3.4

Non-Contiguous addresses after insertion

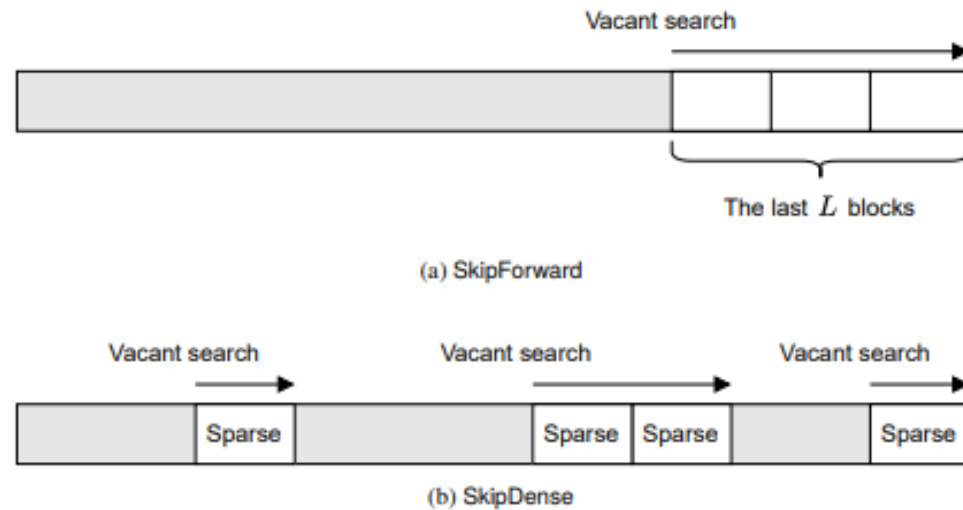


Spatial Locality and Contiguous Memory Allocation. As shown in figure 3.4 in pointer-based trie structures such as the classical Aho-Corasick automaton, individual nodes are allocated separately on the heap as patterns are inserted, resulting in scattered memory addresses. The enhanced AC automaton in this study implements Two-Pass BFS memory layout, which rearrange trie nodes in contiguous memory locations, allowing the CPU prefetcher to predict and preload the next needed memory address before it is executed. This theory covers the concept of cache misses, which occur when data elements are not in fast cache memory and have to be fetched from slower main memory, which addresses the memory reorganization proposed in this study.



Figure 3.5

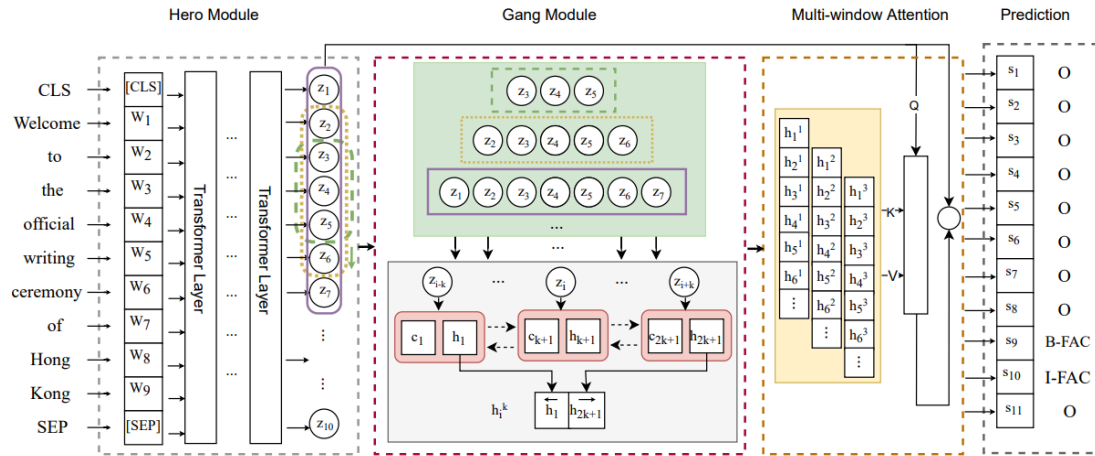
Sparse and Dense Block Classification in Double-Array AC. Adapted from Kanda et al. (2023)



Hash Map and Tiered Storage Theory. A data structure that maps keys to values using a hash function, providing average-case constant time $O(1)$ complexity for insertion, lookup, and deletion operations. In the context of the enhanced AC automaton, the tiered storage strategy leverages hash maps as a secondary storage layer for infrequently accessed or overflow transition states that cannot be efficiently represented in a flat array structure. The baseline Aho-Corasick automaton allocates a full transition table for every state, regardless of how sparse the actual transitions are, resulting in excessive memory consumption especially when the alphabet size is large. The enhanced AC automaton addresses this by implementing a two-tier transition storage approach. As seen in Figure 3.5, a threshold τ is applied to classify each state's transition block as either dense or sparse, densely populated states stay in direct array-indexed transitions for fast $O(1)$ access, while sparsely populated states delegate their transitions to a hash map, significantly reducing memory overhead without sacrificing lookup speed. This structure ensures that the automaton remains both memory-efficient and performant across varying pattern densities and alphabet distributions.

Figure 3.6

Local Context Window Extraction Around a Matched Token. Adapted from Hu et al. (2022)



Context Window Theory. Refers to the principle of examining a bounded region of surrounding text around a detected match to refine or validate the significance of that match within its local linguistic environment. In classical Aho-Corasick implementations, pattern matching operates purely at the lexical level, identifying occurrences of keywords without any consideration of the surrounding words or phrases, which can lead to improper matches and semantically ambiguous detections. The enhanced AC automaton in this study incorporates a context window mechanism that captures a defined span of tokens during and after each matched pattern, enabling post-match contextual analysis to filter out irrelevant matches and improve detection accuracy. As seen in Figure 3.6, this approach mirrors the sliding window sub-sequence construction used in NER, where a fixed window of k surrounding tokens on both sides of a target position is captured and encoded to extract local contextual features (Hu et al., 2022). This theory is grounded in the linguistic concept that the meaning and relevance of a term is often determined not by the term itself but by the words that appear in its immediate vicinity, a principle widely applied in natural language



processing tasks such as named entity recognition and keyword-in-context indexing. By integrating context window evaluation into the matching pipeline, the enhanced automaton moves beyond surface-level string detection toward a more context-aware pattern recognition process.

3.4.2 Methods and Tools

Objective 1: Eliminating Repeated Failure Link Chain Traversal via Precomputed Skip Table.

To address the Repeated Failure Link Chain Traversal during search, the algorithm transitions from a non-deterministic approach to a fully deterministic finite automaton. The skip table construction in Phase 5 is the most significant departure from the classical Aho-Corasick algorithm. In the baseline algorithm, failure links are built at construction time but are only followed at search time; meaning that for every character in the input text, the search loop may need to repeatedly traverse a chain of failure links before landing on a valid transition. In the worst case, this means multiple pointers are invalid per character, each potentially jumping to a distant memory location. The skip table eliminates this entirely by resolving every possible pair (state, character) once during construction. For every state s and every character a in the alphabet, the table stores the exact destination state directly, whether that answer came from a direct transition or from simulating the failure chain. For clinical applications, this mechanism guarantees that the processing speed remains consistent and fast, regardless of the complexity of the medical terminology or the total length of the prescription records.

Python serves as the primary backend language logic that will be used for this objective. The construction logic will be implemented in Python 3.8 or higher, as specified in the software requirements and Visual Studio Code is used as the integrated development environment. The skip table is represented as a two-dimensional Python dictionary of the form `skip[state][symbol]` to allow fast insertion and lookup.



Once the automaton is built, the skip table is loaded in the Flask app for it to stay in the memory and doesn't need to be rebuilt for every new search that is done. This is important in clinical use, where prescription images are processed one after another. The following pseudocode illustrates how the transition table is populated to prevent repeated traversals.

Pseudocode for construction of precomputed Skip Table

1. *For each state s :*
 - a. *For each character a in the alphabet:*
 - i. *If s already has a direct transition on a :*
 1. $skip[s][a] = \text{that transition}$ // fast path
 - ii. *Else:*
 1. *Walk s 's failure chain until we find a state with a transition on a*
 2. $skip[s][a] = \text{wherever that chain lands}$ // slow path, done once only
2. *Return the finished automaton*

The practical consequence at search time is distinct. In classical Aho-Corasick, the inner search loop contains a *while* loop that follows failure links until a valid transition is found, making the per-character cost variable and cache-unfriendly since failure links can point anywhere in the automaton. With the skip table, the inner loop collapses to a single array lookup $state = skip[state][a]$ with no branching and no pointer chasing. The trade-off is memory, the skip table requires $|Q| \times |\Sigma|$ entries, which can be substantial if the alphabet is large and the automaton has many states. This is precisely why Phase 3's hot/cold classification matters, hot states with many real transitions justify a full flat array, while cold states use a hash map to avoid allocating a full row in the skip table for states that would be mostly empty. Together, Phases 3 and 5 convert Aho-Corasick's variable-cost search loop into a uniform $O(1)$ -per-character scan.



By implementing this solution, the algorithm eliminates most problems typically associated with traditional string matching. The precomputed nature of the transitions allows the search phase to operate with high precision, specifically aiding in the identification of medical terms that may vary slightly in spelling or formatting. This structural improvement ensures that the system is not only faster but also more robust when handling the noisy, diverse, and high-volume data found in modern healthcare environments.

Objective 2: Enhancing Memory Access Efficiency via Two-Pass Breadth-First Search Layout

OTO address the Scattered Non-Contiguous Node Allocation causing Cache Misses, the algorithm incorporates a memory reorganization strategy centered on spatial locality. By incorporating the two-pass BFS, the system relocates all trie nodes into a single contiguous memory block immediately following the construction phase. This ensures that nodes sharing common medical prefixes are physically adjacent in memory, rather than being spread across different addresses. This structural alignment is crucial for handling large medical dictionaries, as it minimizes the distance the CPU must travel to retrieve related data, thus optimizing performance during intensive search tasks.

This reorganization directly optimizes how hardware interacts with the software by enabling the CPU prefetcher to operate at peak efficiency. When nodes are stored sequentially, the processor can accurately predict and preload the next required memory address into the cache before it is explicitly requested. This proactive data loading significantly reduces cache miss rates, which is vital when navigating thousands of complex medical terms, abbreviations, and local variants. To implement this hardware-aware optimization, the following logic is integrated into the build process to transition the trie into an optimized memory state.

For the clinical pattern set used in this study, the Two-Pass BFS layout proves to be especially effective because medical terminology is prefix-heavy: drug name prefixes such as "amox-", "metro-", and "cefi-" each produce dense subtrees that land



in contiguous memory blocks after BFS ordering. The following pseudocode illustrates the two-pass reorganization as implemented.

Pseudocode for Two-Pass Breadth First Search (BFS) Traversal

1. *Start a queue with just q_0*
2. *While the queue is not empty:*
 - a. *Take the next node v off the queue*
 - b. *Record v 's position in bfs_order*
 - c. *Add all of v 's children to the queue (in character order)*
3. *For each node v in bfs_order :*
 - a. *For each transition (character $a \rightarrow$ child node):*
 - i. *Replace the pointer with the child's index in bfs_order*
4. *Store all nodes in a flat array in BFS order*

The two-pass BFS layout in Phase 2 addresses a subtle but important performance problem that the baseline Aho-Corasick algorithm ignores entirely. In the classical algorithm, trie nodes are typically allocated individually on the heap as patterns are inserted, meaning nodes that share a common prefix (for example, "cardio" and "cardiovascular") may end up scattered across memory with no spatial relationship to each other. When the search traverses these nodes at runtime, each pointer risks a cache miss because the next node could be anywhere. The first pass of the BFS here solves this by visiting every node in breadth-first order and recording its rank, meaning nodes closest to the root get the lowest indices, and nodes that share prefixes naturally end up adjacent in the ranking.

The second pass then commits to this layout by rewriting every child pointer from a heap address into a flat array index, after which all nodes are stored together in a single contiguous allocation. This is where the approach diverges most sharply from the baseline, classical Aho-Corasick leaves nodes as individually allocated objects connected by pointers, so traversing a path through the automaton is a sequence of random memory accesses. After the two-pass BFS, traversing a path instead walks forward through a flat array, which is exactly the access pattern that



CPU prefetchers are designed to accelerate. The cost is a one-time $O(|Q|)$ remapping pass at build time, and the constraint that the array is immutable after construction, adding new patterns requires a full rebuild. For a static clinical dictionary that is compiled once and queried many times, this is a worthwhile trade.

By moving away from randomized node allocation, the algorithm ensures that the search phase remains fluid and uninterrupted by high-latency memory retrievals. The resulting memory layout is not only more organized but also specifically tuned for the high-speed requirements of modern medical record systems. Ultimately, this approach transforms the trie from a simple data structure into a cache-optimized engine capable of processing high volumes of clinical information with minimal hardware overhead. The resulting memory layout is optimized for fast performance, making it suitable for modern medical record systems. It is designed to work well with the required hardware, such as at least an Intel Core i5 (11th Gen) processor. Using an SSD instead of an HDD also helps improve performance by reducing the time needed to load the data into memory.

Objective 3: Optimizing Memory Consumption via Tiered Hot/Cold Transition Storage

To address the Full Alphabet Transition Storage per state regardless of actual usage, the algorithm adopts a Tiered Hot/Cold Transition Storage strategy. This approach evaluates the density of transitions for each individual state in the trie to determine the most efficient storage format. By classifying states as either "hot" or "cold," the system avoids the full-alphabet overhead, which often leads to significant memory waste in sparse sections of the dictionary. This targeted allocation ensures that memory resources are focused only where they are truly needed, creating a more compact and streamlined data structure.

This tiered storage model significantly reduces the overall memory footprint of the medical dictionary, which is critical given the minimum hardware specification of 8 GB RAM stated in Section 3.3.1. States with numerous transitions, typically those near the root of the trie representing common prefixes, are stored in flat arrays



for rapid constant-time access. In contrast, deeper nodes with fewer transitions are stored in hash maps to save space. The freed memory headroom also supports the context-awareness layer described below, which requires additional runtime structures such as the ambiguous terms set and the positive.negative context token windows. The following pseudocode illustrates the Tiered Hot/Cold classification as implemented.

Pseudocode for adopting a Tiered Hot/Cold Transition Storage strategy

1. *For each node v :*
 - a. *If v has $\geq \theta$ outgoing transitions: // "hot" — worth the memory cost*
 - i. *Store v 's transitions as a flat array over the whole alphabet*
 - b. *Else: // "cold" — too sparse for a flat array*
 - i. *Store v 's transitions as a hash map*

The tiered hot/cold classification in Phase 3 is a direct response to the memory inefficiency that arises when a flat transition array is applied uniformly across all states, which is the approach taken in the most common optimized variant of Aho-Corasick. In that baseline, every state allocates a full array of size $|\Sigma|$, so that any transition can be resolved in $O(1)$ by direct indexing. This works well for states near the root that see many different characters, the root of a clinical trie, for instance, likely has transitions for nearly every letter since patterns begin with a wide variety of characters. But deeper states, representing longer prefixes like "cardiov-" or "hyperinsu-", realistically transition on only a handful of characters. Allocating a full alphabet-sized array for these sparse states wastes memory proportional to $|\Sigma|$ minus the actual number of transitions, and in a large clinical dictionary with a broad alphabet this waste accumulates significantly across thousands of deep nodes. The CSV/RRF dataset files containing drug names, dosages, frequencies, abbreviations, and treatments are loaded into the trie at build time using Python's built-in csv module, and the hot/cold classification is applied immediately after the Two-Pass BFS layout is finalized.



By implementing this tiered storage model, the algorithm significantly reduces the overall footprint of the medical dictionary in the system's memory. This efficiency allows the application to handle more extensive datasets and complex variations without exceeding hardware limitations. Ultimately, the transition from a rigid storage format to an adaptive one ensures that the algorithm remains scalable and effective for modern healthcare data environments.

Context-Aware Validation and Priority-Weighted Scoring

Once candidate matches are produced by the enhanced Aho-Corasick search engine, two additional processing layers are applied before results are reported. These layers are implemented in Python and operate on the TrOCR-extracted text that has been digitized from handwritten prescription images.

Context-Aware Validation. TrOCR, implemented via the Hugging Face Transformers library with PyTorch as the underlying deep learning engine, converts prescription images into raw text strings. OpenCV (cv2) and Pillow (PIL) are used for image preprocessing prior to TrOCR: OpenCV converts the image to grayscale, applies adaptive thresholding, dilation, and contour detection to isolate handwritten regions, while Pillow handles image loading and format normalization. The raw text produced by TrOCR is then passed into the enhanced AC search engine. For each candidate match returned, the context-aware validation layer extracts a token window of $\pm k$ tokens surrounding the matched position. If the matched term belongs to the AMBIGUOUS_TERMS set — for example, the word 'cold' which could indicate a symptom or a physical descriptor as in 'cold compress' — the surrounding window is checked against a NEGATIVE_CONTEXT token set to suppress false positives and a POSITIVE_CONTEXT token set to confirm valid clinical usage. This logic is implemented as a Python function operating on the tokenized output of TrOCR, ensuring that disambiguation is embedded within the matching pipeline rather than applied as a disconnected post-processing step.



Priority-Weighted Scoring. Validated matches are then scored using a priority-weighted mechanism that resolves overlapping candidates. The scoring function is implemented in Python and computes a normalized score in the range [0, 1] based on four additive components: a length bonus that rewards longer, more specific matches; a word boundary bonus that confirms the match aligns with token boundaries in the TrOCR output; a known-term bonus that elevates terms confirmed present in the high-priority CSV/RRF medical dictionary; and a context bonus that rewards matches confirmed valid by the context-aware layer. Validated matches are sorted by descending priority score, overlaps are resolved by retaining the highest-scoring non-overlapping hit for each text position, and the final results are served through the Flask REST API to the HTML/CSS web interface, which displays color-coded annotations, a structured medical terms table categorized by type and match confidence (exact or fuzzy), and an abbreviations panel mapping shorthand notations to their full clinical meanings.



REFERENCES

- Negro-Calduch, E., Azzopardi-Muscat, N., Krishnamurthy, R. S., & Novillo-Ortiz, D. (2021). Technological progress in electronic health record system optimization: Systematic review of systematic literature reviews. *International Journal of Medical Informatics*, 152, 104507. <https://doi.org/10.1016/j.ijmedinf.2021.104507>
- Mohan, D. (n.d.). AHO-Corasick Algorithm - CSU083. *Shoolini University*. <https://dmj.one/edu/su/course/csu083/theory/aho-corasick-algorithm>
- Kim, J., Weiss, J. C., & Ravikumar, P. (2022, April 1). Context-Sensitive spelling correction of clinical text via conditional independence. <https://pmc.ncbi.nlm.nih.gov/articles/PMC11044887/>
- Kumar, M. (2025, March 23). Understanding and implementing the AHO-Corasick algorithm for string matching. *upGrad Blog*. <https://www.upgrad.com/blog/aho-corasick-algorithm/>
- Fayez, M., Abbas, A., Khaled, H., & Ghoniemy, S. (2024). Enhanced AHO-Corasick Algorithm for network Intrusion Detection Systems. *International Journal of Intelligent Computing and Information Sciences* 24(3), 83–92. <https://doi.org/10.21608/ijicis.2024.315432.1349>
- Gagniuc, P. A., Păvăloiu, I., & Dascălu, M. (2025). The AHO-Corasick paradigm in modern antivirus engines: a cornerstone of Signature-Based Malware Detection. *Algorithms*, 18(12), 742. <https://doi.org/10.3390/a18120742>



- Kanda, S., Akabe, K., & Oda, Y. (2023). Engineering faster double-array Aho–Corasick automata. *Software Practice and Experience*, 53(6), 1332–1361.
<https://doi.org/10.1002/spe.3190>
- ~~Lai, W., Wu, C., Lai, L., & Sic, M. (2019). Two-Phase PFAC algorithm for multiple patterns matching on CUDA GPUs. *Electronics*, 8(3), 270.
<https://doi.org/10.3390/electronics8030270>~~
- Baloi, A., Belean, B., Turcu, F., & Peptenatu, D. (2023). GPU-based similarity metrics computation and machine learning approaches for string similarity evaluation in large datasets. *Soft Computing*, 28(4), 3465–3477.
<https://doi.org/10.1007/s00500-023-08687-8>
- Çelebi, M., & Yavanoğlu, U. (2023, July 11). Accelerating pattern matching using a novel Multi-Pattern-Matching algorithm on GPU. *Applied Sciences*, 13(14), 8104.
<https://doi.org/10.3390/app13148104>
- Zeitak, A., & Morrison, A. (2021). Cuckoo Trie. *ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP '21)*, 147–162.
<https://doi.org/10.1145/3477132.3483551>
- Samyal, Dr. V. K., Baghla, R., Aaksh, & Chanderkant. (2025). A Cache-Centric Performance Analysis Of Pointer-Based Data Structures: Skiplists Versus Linked Lists. *INTERNATIONAL JOURNAL OF CREATIVE RESEARCH THOUGHTS - IJCRT*, 13(12).
<https://www.ijcrt.org/searchpage.php?searchvalue=A+Cache-Centric+Performance+Analysis+Of+Pointer-Based+Data+Structures%3A+Skiplists+Versus++Linked+Lists>



- Tupsamudre, H., Jain, S., & Lodha, S. (2021, December 4). PhishMatch: a layered approach for effective detection of phishing URLs. arXiv.org. <https://arxiv.org/abs/2112.02226>
- Hussain, M. E., & Hussain R. (2021, December 21). Real time Aho-Corasick Implementation of String Matching technique suitable for smart IOT. *IEEE Conference Publication* | *IEEE Xplore*. <https://ieeexplore.ieee.org/document/9645846>
- Bazoge, A., Morin, E., Daille, B., & Gourraud, P. (2023). Applying natural language processing to textual data from clinical data warehouses: Systematic review. *JMIR Medical Informatics*, 11, e42477. <https://doi.org/10.2196/42477>
- Ngo, D.-H., & Koopman, B. (2024). From free-text drug labels to structured medication terminology with BERT and GPT. *AMIA Annual Symposium Proceedings*, 2023, 540–549. <https://pmc.ncbi.nlm.nih.gov/articles/PMC10785872/>
- Mahajan, D., Liang, J. J., Tsou, C.-H., & Uzuner, Ö. (2023). Overview of the 2022 n2c2 shared task on contextualized medication event extraction in clinical notes. *Journal of Biomedical Informatics*, 144, 104432. <https://doi.org/10.1016/j.jbi.2023.104432>
- Alfattni G, Belousov M, Peek N, Nenadic G. (2021). Extracting drug names and Associated Attributes from Discharge Summaries: Text mining study. *JMIR Medical Informatics*, 9(5), e24678. <https://doi.org/10.2196/24678>
- Fraila Navarro, D., Ijaz, K., Rezazadegan, D., Rahimi-Ardabili, H., Dras, M., Coiera, E., & Berkovsky, S. (2023). Clinical named entity recognition and relation extraction using natural language processing of medical free text: A systematic review.



International Journal of Medical Informatics, 177, 105122.

<https://doi.org/10.1016/j.ijmedinf.2023.10512>

Espinas, E., Galvez, L. A., Merino, M. Y. S. J., Ramos, J. M., & Soriano, R. D. (2025). Real-time CNN-based generic medicine name classification for prescription recognition. *ISU Linker: Journal of Education, Social Sciences and Allied Health*, 1(1). <https://www.isujournals.ph/index.php/jessah/article/view/220>

Benitez, R. A. A., Acula, D. D., Bondoc, A. O. M., Hizon, A. L. L., & Santos, A. J. D. (2024). Transcribing handwritten medical prescription using Convolutional Neural Network AlexNet architecture and Canny Edge Detection. *Green Intelligent Systems and Applications*, 4(1), 41–53. <https://doi.org/10.53623/gisa.v4i1.417>

Robantes, J. L., Serrano, A., Dioses, R., & Cortez, D. M. (2024). Enhanced named entity recognition algorithm for Filipino cultural and heritage texts. *World Journal of Advanced Research and Reviews*, 24(3), 2177–2186. <https://doi.org/10.30574/wjarr.2024.24.3.3905>

Ashok, A., Ravichandran, R., Paul, R., Vimal, C., & Belwal, M. (2024). Advancements in String-Searching Algorithms: A Novel Approach. *Proceedings of the 15th International Conference on Computing, Communication and Networking Technologies (ICCCNT)*. <https://ieeexplore.ieee.org/document/10724792>

Pino, R., & Mendoza, J. L. (2025). Optical Recognition of the Philippines' Ancient Text: A Deep Learning Approach. *International Journal of Multidisciplinary Research and Analysis*, 8(3), 1266-1272. https://www.researchgate.net/publication/390103091_Optical_Recognition_of_the_Philippines'_Ancient_Text_A_Deep_Learning_Approach

Tinam-isan, M. A. C., & Naga, J. F. (2024). Exploring the Landscape of Health Information Systems in the Philippines: A Methodical Analysis of Features and Challenges. *Journal of Health Informatics*.



<https://journal.uob.edu.bh/server/api/core/bitstreams/d58f6ce5-c0c9-4c7c-9df9-83268fb89dd9/content>

- Gor, A., & Bhensdadia, C. K. (2025). Comparative Analysis of Classical String Matching Algorithms with Insights into Applications, Parallel Processing, and Big Data Frameworks. *IJCA*, 187(53).
<https://www.ijcaonline.org/archives/volume187/number53/comparative-analysis-of-classical-string-matching-algorithms-with-insights-into-applications-parallel-processing-and-big-data-frameworks/>
- Falcon, R. M. G., Alcazar, R. M. U., Babaran, H. G., Caragay, B. D. B., Corpuz, C. A. A., Kho, M. V. S., Perez, A. C. N., & Isip-Tan, I. T. C. (2024). Exploring Filipino Medical Students' Attitudes and Perceptions of Artificial Intelligence in Medical Education: A Mixed-Methods Study. *MedEdPublish*, 14, 282.
<https://doi.org/10.12688/mep.20590.2>
- Elepaño, I. T. G., Amit, A. M. L., Cabaluna, I. T. G., Sevilla, S. F., Carandang, T. H. D. C., Espiritu, A. I., & Tan-Lim, C. S. C. (2025). Implementing electronic health records in Philippine primary care settings: Mixed-methods pilot study. *JMIR Medical Informatics*, 13(1). <https://medinform.jmir.org/2025/1/e63036>
- Zuniga, R. A., Magtubo, K. M., Marcelo, A., Sy, M., Alcantara-Zuniga, M. A. R., & Sison, O. T. (2026). Development and implementation of an interoperable local health information exchange platform for the attainment of universal health coverage in the Philippines. *Journal of Medical Internet Research*, 14.
<https://pmc.ncbi.nlm.nih.gov/articles/PMC12915754/>
- Montalan, J. J., Adarlo, M. A., Decano, A., & Oco, N. (2025). Batayan: A Filipino NLP benchmark for evaluating Large Language Model. *arXiv preprint*.
<https://arxiv.org/html/2502.14911v1>
- De Mesa, R. A., Magtubo, K. M., Marcelo, A., Sy, M., Alcantara-Zuniga, M. A. R., & Sison, O. T. (2024). Facing the digital frontier: exploring user acceptance of



electronic health records in an urban, rural and remote setting in the Philippines.

BMJ Open Quality. 2024;13:e002621.

<https://doi.org/10.1136/bmjopen-2023-002621>

Caballes, A. B. (2021). Acceptability of Information Technology Systems Developed for Distant Philippine Communities among Local Health Providers. *Acta Medica Philippina*, 55(1).

<https://actamedicaphilippina.upm.edu.ph/index.php/acta/article/view/2815>

~~Hládek, D., Staš, J., & Pleva, M. (2020). Survey of automatic spelling correction. *Electronics*, 9(10), 1670. <https://doi.org/10.3390/electronics9101670>~~

Chakraborty, S., Grossi, R., Sadakane, K., & Satti, S. R. (2023). Succinct representation for (non)deterministic finite automata. *Journal of Computer and System Sciences*, 131, 1–12. <https://doi.org/10.1016/j.jcss.2022.07.004>

Hu, J., Shen, Y., Liu, Y., Wan, X., & Chang, T.-H. (2022). Hero-Gang neural model for named entity recognition. *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 1924–1936. <https://aclanthology.org/2022.naacl-main.140>

Gholipour, M., Khajouei, R., Amiri, P., Hajesmaeel Gohari, S., & Ahmadian, L. (2023). Extracting cancer concepts from clinical notes using natural language processing: A systematic review. *BMC Bioinformatics*, 24(1), 405. <https://doi.org/10.1186/s12859-023-05480-0>

Durango, M. C., Torres-Silva, E. A., & Orozco-Duque, A. (2023). Named entity recognition in electronic health records: A methodological review. *Healthcare Informatics Research*, 29(4), 286–300. <https://doi.org/10.4258/hir.2023.29.4.286>

Karthikeyan, S., de Herrera, A. G. S., Mirza, A., & Doctor, F. (2021). An OCR post-correction approach using deep learning for processing medical reports.



IEEE Transactions on Circuits and Systems for Video Technology, 32(2), 792–802. <https://doi.org/10.1109/TCSVT.2021.3087641>

Liu, S., Wang, A., Xiu, X., Zhong, M., & Wu, S. (2024). Evaluating medical entity recognition in health care: Entity model quantitative study. *JMIR Medical Informatics*, 12, e59782. <https://doi.org/10.2196/59782>

Kate, R. J. (2021). Clinical term normalization using learned edit patterns and subconcept matching. *JMIR Medical Informatics*, 9(1), e23104. <https://doi.org/10.2196/23104>

Redwan, K., Ahmed, M., Al Sohan, M. F. A., Munna, M. M. H., Akter, S. S., Emon, M. A. S., & Dutta, A. (2024). Optimizing big data pattern search: Advanced string manipulation techniques in C++. 3rd International Conference on Computing Advancements (ICCA '24). ACM. <https://doi.org/10.1145/3723178.3723194>

Khancome (2026). Multiple string pattern matching algorithm using multi-character inverted lists. *Algorithms*, 19(5), 362. <https://www.mdpi.com/1999-4893/19/5/362>